GIT WORKSHOP

Wolfgang Höfer

Vorbemerkung

Git ist das aktuell wohl gängigste Versionskontrollsystem auf dem Markt. Allerdings gibt es im Zusammenhang mit *git* so viele verschiedene Aspekte zu betrachten, dass eine stringente Einführung in das Thema relativ schwierig ist. In diesem Workshop kümmern wir uns schwerpunktmäßig um folgende Punkte:

- Installation (lokal und auf dem Server)
- Arbeiten als Single-Developer
- Arbeiten im Team (Grundlagen)
- Durchspielen von Anwendungsszenarien
- CI/CD und TDD auf ganz einfachem Niveau

Für den reinen Client-Aspekt ist die Verwendung von *git* auf Windows problemlos. Sobald es aber um einen *git*-Server geht, sind bessere Systeme wie Linux oder MacOS sinnvoller[1] Wir werden uns also mit beiden Welten (Windows und Linux) beschäftigen müssen. Wer hier noch keine Erfahrung hat, wird viel profitieren!

[1] Ich habe bisher noch keinen Server im Internet getroffen, auf dem MacOS als Sytem verwendet wird. Das kann sich aber natürlich jederzeit ändern!

1 Installation

1.1 Vorabinformation

Die Installation auf dem Client (eigener Rechner mit Windows, Linux, MacOS) ist im Regelfall völlig problemlos und wird nachfolgend noch genau beschrieben. Für einen Git-Server braucht man – wie schon der Name sagt – einen Server. Da es für einfache Versuche etwas aufwändig ist, sich eine entsprechende Installation im Internet zu mieten, greifen wir in diesem Workshop auf virtuelle Umgebungen zurück. Damit ist gemeint, dass sich jeweils 4 Teilnehmer ein Gerät teilen, das für sie jeweils einen eigenen Spielserver bereit stellt. Die Installation dieser Infrastruktur ist im Anhang genauer beschrieben. Es ist natürlich auch möglich, einen Spielserver mit Hilfe von Virtualbox auf dem eigenen Computer zu installieren. Der eigentliche Server wird in jedem Fall ein Ubuntu 24.04 sein.

Wem die letzten Zeilen gar nichts gesagt haben, der sollte im Anhang den Abschnitt über Virtualisierung lesen und dann hierher zurückkehren.

1.2 Client-Installation

Für die ersten Schritte mit *git* ist es sehr sinnvoll, einen ganz einfachen Client zu verwenden. Hier ist mit *einfach* nicht die Bedienung sondern eher *ursprünglich* gemeint. Mittlerweile bringt jede Entwicklungsumgebung bereits eine eigene Version eines *git*-Clients mit und wir können beim besten Willen nicht jede Variante, eventuell noch mit eigenen Features, besprechen.

Faktisch gibt es eigentlich keinen reinen *git*-Client, weil immer automatisch Client und Server installiert werden. Gerade bei Windows bedeutet das aber nicht, dass man den Server auch verwenden kann. Hier hat Microsoft so viele Hindernisse in den Weg gelagt, dass man besser die Finger davon lässt!

1.2.1 Windows

Für die Installation auf Windows ist es hilfreich, wenn du dich an folgende Reihenfolge hältst:

- Notepad++ installieren
- Git installieren

1.2.1.1 Notepad++

Lade von notepad-plus-plus.org den Editor *notepad++* herunter und installiere das Programm durch den üblichen Doppelklick. Es gibt keine Stellen, wo das Vorgehen nicht eindeutig wäre. Du wirst später bei der Installation von *git* nach diesem Programm gefragt werden!

1.2.1.2 Git

Lade von gitforwindows.org die Installationsdatei herunter.

Starte nun die Installation von *git* durch Doppelklick auf die Installationsdatei. Die nachfolgenden Bilder zeigen den Ablauf in der Version vom November 2025.

ToDo: Bilderstrecke

1.2.2 Linux

Hier hängt es etwas von der Distribution ab. Bei Ubuntu genügt der Befehl `apt install git -y`

1.2.3 MacOS

MacOS ist ähnlich pflegeleicht. Mit `brew install git` ist die Insallation ebenfalls bereits gestartet und nach kurzer Zeit beendet.

1.3 Serverinstallation

Dieser Abschnitt ist nun deutlich umfangreicher und kann bei einer ersten Lektüre auch problemlos übersprungen werden, wenn du dich nur für die Verwendung von *git* auf deinem eigenen, isolierten Rechner interessierst.

Ansonsten wirst du nachfolgende Schritte durchlaufen:

- Allgemeine Gedanken zum Serverbetrieb
- Anmelden auf dem späteren *git*-Server
- Installation eines *normalen git*
- Docker kennenlernen
- GitLab mit Docker installieren

Die Bedienung von GitLab lernst du später im Workshop noch genauer.

1.3.1 Allgemeine Gedanken

Der Betrieb eines Servers ist auch immer mit Verantwortung und Aufwand verbunden. Wer hier keine Kenntnisse besitzt, schreckt schnell davor zurück, obwohl die Grundlagen eigentlich relativ schnell gelernt sind. Anleitungen und Checklisten aus dem Internet erleichtern hier auch den Einstieg.

Generell muss man sich natürlich über den Einsatzzweck des Servers Gedanken machen. Was für Dienste soll er bereitstellen? Müssen Benutzer verwaltet werden, was sollen diese Benutzer dürfen. Es liegt in der Natur der Sache, dass dich das Anwendungsspektrum im Lauf der Zeit vergrößert und damit wächst zwangsläufig auch das Wissen.

Eine besondere Hürde stellt der Betrieb von mehreren Webangeboten auf dem gleichen Server dar, die über verschiedene Namen angesprochen werden sollen (homepage.schule.de, gitlab.schule.de, versuche.schule.de, …). Die Lösung hierfür ist ein *reverse Proxy*, allerdings sprengt das den Rahmen dieses Workshops.

1.3.2 Anmelden auf dem Server

Damit du dich auf deinem Server anmedlden kannst, benötigst du folgende Informationen:

IP-Adresse des Servers Benutzername Kennwort

Diese Daten werden dir im Workshop mitgeteilt. Solltest du mit deiner eigenen Installation arbeiten, so sollten dir diese Informationen bereits vorliegen.

Die Anmeldung auf dem Server erfolgt über das Protokoll *ssh*.
Seit einer der letzten Versionen von Windows ist auch hier bereits ein ssh-Client eingebaut. Bei Lunx und MacOS gehört er seit Jahrzehnten zum Standard. Ich selbst bevorzuge auf Windows allerdings eher das (kostenlose) Programm *Putty* und die gerade vorher installierte *git*-bash.

In jedem Fall musst du zunächst ein Terminal (Kommandofenster) öffnen um den nachfolgenden Befehl eingeben zu können:

```
ssh benutzer@IP-adresse
```

Du erhältst eine Sicherheitsabfrage um zukünftige Man-in-the-middle Angriffe erkennen zu können, die du mit *yes* beantwortest. Bei der nachfolgenden Kennworteingabe kann es (je nach System) sein, dass die Tastendrücke nicht angezeigt werden!

Wenn alles geklappt hat, so wirst du vom Eingabepromt des Terminals auf dem Server begrüßt:

```
benutzer#>
```

1.3.3 Installieren von Git

Vor jeder Installation muss die Softwaredatenbank aktualisiert werden, damit keine veralteten Dateien installiert werden. Als normaler Benutzer darfst du das aber nicht und musst zuerst zum Administrator (=root) werden:

```
sudo -i
```

Es wird wieder dein Kennwort abgefragt und der Eingabeprompt ändert sich. Nun kannst du administrative Arbeiten erledigen.

```
apt update
```

In regelmäßigen Abständen sollte auch ein Komplettupgrade des Systems laufen, um z.B. Sicherheitslücken zu schließen und *alle* Programme zu aktualisieren[1] Da auf deinem Server eine komplett neue Installation von DVD ist, kann einiges veraltet sein – also führst du aus:

```
apt upgrade -y
```

Das kann nun einige Zeit dauern. Hättest du eine weitere Verbindung geöffnet, so könntest du parallel weiterarbeiten, aktuell warten wir aber ab.

Nun zu eigentlichen Installation von *git*:

```
apt install git -y
```

Damit ist die Installation von *git* bereits beendet, solange keine besondere Zusatzsoftware benötigt wird, die auf dem Server ohnehin unüblich ist.

1.3.4 Absicherung

Gleich an dieser Stelle ein Wort zur Sicherheit. Wenn dein Server im Internet steht, dann bietet er natürlich eine große Angriffsfläche. Es ist deshalb sinnvoll, wenn du zumindest die Firewall aktivierst und ein sicheres Kennwort einstellst. Das hat im Prinzip nichts mit *git* zu tun, ist aber dennoch wichtig.

1.3.4.1 Kennwort

Das geht relativ schnell. Gib einfach im Terminal den Befehl `passwd` ein und drücke `Enter`. Du wirst zweimal nach einem neuen Kennwort gefragt und damit ist bereits alles erledigt.

1.3.4.2 Firewall

In Ubuntu ist eine sehr pflegeleichte Firewall integriert (UFW). Du gehst nun in 2 Schritten vor:

- Definieren des/der offenen Ports (bei uns 22 für SSH)
- Aktivieren der Firewall

```
ufw status verbose
# sollte *inactive* liefern

# Die Firewall darf hier noch nicht aktiv sein !!!!!!!
ufw default deny incoming
ufw default allow outgoing
```

[1] vgl. Windows … hier muss jedes Programm einzeln aktualisiert werden!

```
# ssh erlaben
ufw limit 22/tcp

# Firewall einschalten
ufw enable
ufw status verbose
```

Hinweise
An Stelle von *limit* könnte auch *allow* stehen. Allerdings blockert *limit* den Zugang für eine bestimmte Zeit, wenn zu viele gescheiterte Anmeldeversuche in kurzer Zeit vorliegen.

1.3.4.3 Probleme mit Ports

Wir werden im weiteren Verlauf ein Problem mit Port 22 bekommen. Die Software GitLab, die wir später installieren werden, beansprucht den Port 22 (sinnvoller Weise) für sich und das führt zu einer Kollision mit dem SSH-Dienst des Servers. Wir werden ihn zu gegebener Zeit auf Port 23 umlegen.

1.3.5 Wer darf was?

Das ist nun eine Grundsatzdiskussion. Wenn du in dieser Konfiguration mit den Schülern *git* verwenden willst, müssen die Schüler auch Zugriff auf den Server haben. Es ist zwar generell möglich, Daten anonym vom Server abzuholen (=clone, pull), aber ein Speichern ist dann nicht möglich (push). Aus diesem Grund muss für die Schüler eine Zugriffsmöglichkeit geschaffen werden. Wenn wir später mit GitLab arbeiten, dann ist das komplett anders!

Der einfachste – aber für alle Beteiligten unkomfortabelste – Weg ist für alle Beteiligten das anlegen eines echten Benutzeraccounts mit Kennwort[2].

Wir werden das System so einrichten, dass alle Benutzer mit dem gleichen Login arbeiten. Das klingt auf den ersten Blick gefährlich oder fahrlässig, aber wir sichern das natürlich ab!

An dieser Stelle trifft Theorie auf Praxis. Damit die Schüler einen Zugriff auf den Server bekommen (eigentlich nur auf *git*) ohne das Kennwort des gemeinsamen Benutzers kennen zu müssen, verwenden wir RSA, d.h. jeder Schlüssel braucht einen privaten und einen öffentlichen Schlüssel. Die öffentlichen Schlüssel der Schüler werden vom Lehrer als Berechtigt eingetragen.

1.3.5.1 Schlüssel erzeugen

Das dürfte in dem gesamten Prozess die Schlüsselstelle sein, da hier die Schüler eigenständig tätig werden müssen. Ein zusätzliches Problem ist, dass die Schüler an unterschiedlichen Geräten arbeiten – zuhause, vielleicht je nach Stundenplan in unterschiedlichen Computerräumen, … und auf jedem Computer muss die Schlüssel-Infrastruktur eingerichtet sein.

Ich würde hier gerne auf GitLab vertrösten, aber um dieses Problem kommen wir nicht herum, da auch in GitLab der Public-Key hinterlegt werden muss und der private Schlüssel auf dem Client vorhanden sein muss. Die einzige Abhilfe ist eine konsequente Verwendung der

[2] Das ist durchaus sinnvoll, wenn mit den Schülern auch andere Techniken als *git* geübt werden sollen!

Weboberfläche als Editor, aber das ist eben keine Java-IDE. Ein Weg – wenn auch unsicher – ist das Mitführen des Schlüsselpaares auf einem USB-Stick (oder Cloud-Storage). Der private Schlüssel muss dann eben am Anfang der Stunde an die richtige Stelle kopiert werden. Noch unsicherer aber einfacher wäre es, wenn alle Schüler einfach das gleiche Schlüsselpaar für den Unterricht verwenden. Dass die Idee hinter der asymmetrischen Verschlüsselung damit ad absurdum geführt ist, versteht sich von selbst!

Die Schlüsselerzeugung funktioniert im Prinzip auf allen Betriebssystem gleich (vom Befehl) nur die Kommandozeile muss passend gewählt werden. Auf Windows sollte einfach die *git*-bash verwendet werden, auf den anderen Systemen das Terminal.

```
ssh-keygen -t rsa -b 4096
```

Sollte bereits ein Schlüsselpaar existieren, so bietet sich diese Variante des Befehls an, um den anderen Schlüssel nicht zu überschreiben:

```
ssh-keygen -t rsa -b 4096   -C "gitkey_susi_sandmann@schule.de"
```

Als Lehrer musst du die öffentlichen Schlüssel *einsammeln* (Mail, Cloud, …). Sie sind an der Endung *.pub* erkennbar. Hier laufen wir in das übliche Windowsproblem, dass je nach Konfiguration die Dateiendungen ausgeblendet sein können!

(Im Explorer: Anzeige - Dateinamenserweiterungen anzeigen)

Im Prinzip genügt in vielen Fällen ein kürzerer Schlüssel, aber einige Plattformen (z.B. Gitea) erwarten diese Schlüssellänge.
Je nach perönlichem Vertrauen kann auch ein Online-Schlüsselgenerator verwendet werden:

https://cryptotools.net/rsagen
(Die *einige Sekunden* sind durchaus deutlich mehr Sekunden, wenn man eine Schlüssellänge von 4096 Bit wählt!)

Der Inhalt der (öffentlichen) Schlüsseldateien müssen dann in einer Datei mit dem Namen *authorized_keys* gesammelt und im Ordner des *git*-Benutzers abgelegt werden.

1.3.5.2 Schlüssel eintragen

Der einfachste Weg besteht wohl darin, sämtliche Schlüsseldateien nach Einsendeschluss gesammelt auf den Server zu kopieren und sie dann durch ein kleines Script in die *authorized_keys* zu übertragen.

Schritt 1: Benutzer anlegen

```
adduser --system --shell /usr/bin/git-shell --home /home/git --group --disabled-password git
```

Schritt 2: Dateien übertragen
Hier kommt das Protokoll SCP zum Einsatz. Unter Windows ist es sinnvoll, das grafische Programm *WinSCP* (kostenlos, keine Installation nötig für portable Edition) zu verwenden.

`https://winscp.net/download/WinSCP-6.5.4-Portable.zip/download`

Nach dem Aufruf hat man eine Oberfläche, die an den Explorer erinnert. Nach der Anmeldung über ssh auf dem Server ist die eine Fensterhälfte der lokale Rechner, die andere Fensterhälft der Server. Dateien können über Drag and Drop kopier werden.

Kopiere die '*pub* Dateien in den Ordner */home/git*.

Schritt 3: Dateien zusammenführen
Gib auf der Kommandozeile des Servers folgende Befehlsabfolge ein:

```
cd /home/git
ls *.pub   # Nur zur Kontrolle, ob die Dateien da sind.

for i in $(ls *pub); do cat $i >> .ssh/authorized_keys; done
```

Hinweis
Dieses Script ist **nur** für einen Erstimport geeignet. Werden später noch Schlüsseldateien nachgeliefert, so würden in der *authorized_keys* lauter Duplikate entstehen! Wenn du nach dem Import die Schlüsseldateien löschst, dann können spätere Dateien aber schon auf diese Weise eingespielt werden!

Löschen der Dateien

```
rm *.pub
```

Ab jetzt steht einer Verwendung von *git* über die Kommandozeile nichts mehr im Weg.

2 Lokale Arbeit

Bevor du wirklich mit dem Server arbeiten wirst, steht die lokale Anwendung von *git* im Vordergrund. Du arbeitest also auf deinem Laptop, Arbeitsplatz, …

Du kannst diese Methode verwenden um alleine Software zu entwickeln, aber auch für die Verwaltung von verschiedenen Versionen deiner Unterrichtsvorbereitung.

Nachfolgend besprechen wir folgende Punkte

- Grundkonfiguration (einmalig)
- Erstellen eines Repositories
- Versionierung von Dateien
- Einfache Fehlerbehebung

3 Konfigurieren von git

Öffne die *git*-Bash[1]. Da es sich um globale Einstellungen handelt, muss das nicht unbedingt im Projektordner sein![2].

Aktuelle Einstellungen ansehen

```
# Eingabe
git config --list

# Ausgabe
user.email=
user.name=
     ...
```

Einstellungen anpassen

```
# eigene Phantasiedaten verwenden
git config --global user.name="Susi Sandmann"
git config --global user.email="susi@sandmann.de"
```

3.1 Erstellen eines Repositories

Die Voraussetzung für die Arbeit mit *git* ist ein Projektordner, in dem die relevanten Dateien liegen sollen. In diesem Ordener wirst die dein erstes (lokales) Repository erstellen.

3.1.1 Ordner anlegen

Ich gehe davon aus, dass das Erstellen eines Ordners mit dem üblichen Werkzeugen kein Problem darstellt und zege es hier mit Hilfe des Terminals bzw. der *git*-Bash.

```
mkdir hands_on_1
cd hands_on_1
```

Um den Ordner auch später einfach wiederfinden zu können, sollten wir den Pfad zum Ordner identifizieren und eventuell notieren:

```
pwd
```

[1] bzw. das Terminal deines Systems. Das setze ich in Zukunft stillschweigend voraus
[2] Sollte dir die Schrift im Fenter zu klein sein, kannst du sie mit strg + + vergrößern.

Da auch die *git*-Bash eine Form von abgespecktem Linux darstellt, gelten die nachfolgenden Befehle auf allen drei Systemen in gleicher Weise. Es sinnvoll, wenn du dir diese Liste bereitlegst (oder die Befehle lernst).

Wichtige Befehle

Befehl	Kurzerklärung
`ls -la`	Auflisten aller Dateien im Ordner
`cd`	Wechsel in den Home-Ordner
`cd <ordner>`	Wechsel in *Ordner*
`cd ..`	Wechsel eine Ebene nach oben
`echo <Text>`	Ausgabe von Text
`> <datei>`	Ausgabe in Datei (s.u.)
`mkdir <ordner>`	Ordner erstellen
`rm -rf <ordner>`	Ordner ohne Rückfrage löschen
`cat <datei>`	Inhalt der Datei komplett ausgeben
`less <datei>`	Langsame Ausgabe, Ende mit `q`

TODO: Übungsblatt Shell-Befehle erstellen

TIPP
Im Terminal kann mit der *Pfeil hoch* Taste durch die letzten Befehle geblättert werden – damit gehen manche Dinge sehr schnell!

3.1.2 Anlegen des Repositories

Im Projektordner gibst du folgenden Befehl ein:

```
git init
```

Es erscheint eine Meldung, die je nach System etwas anders aussehen kann:

```
Hinweis: Als Name für den initialen Branch wurde 'master' benutzt. Dieser
Hinweis: Standard-Branchname kann sich ändern. Um den Namen des initialen Branches
Hinweis: zu konfigurieren, der in allen neuen Repositories verwendet werden soll und
Hinweis: um diese Warnung zu unterdrücken, führen Sie aus:
Hinweis:
Hinweis:    git config --global init.defaultBranch <Name>
Hinweis:
Hinweis: Häufig gewählte Namen statt 'master' sind 'main', 'trunk' und
Hinweis: 'development'. Der gerade erstellte Branch kann mit diesem Befehl
Hinweis: umbenannt werden:
Hinweis:
Hinweis:    git branch -m <Name>
Leeres Git-Repository in /tmp/hands_on_1/.git/ initialisiert
```

Es hat sich eingebürgert[3], den Arbeitszweig von *master* auf *main* umzubenennen:

[3] Bedingt durch gesellschaftspolitische Strömungen, die *Master* mit Sklaverei gleichsetzen, aber Eine Master-Arbeit hier nicht als Problem sehen.

```
git branch -m main
```

Den Erfolg kann man schnell mit

```
git status
```

Es sollte folgender Text (eventuell englisch) erscheinen:

```
Auf Branch main

Noch keine Commits

nichts zu committen (erstellen/kopieren Sie Dateien und benutzen
Sie "git add" zum Versionieren)
```

Hinweis
Man kann *git* auch so konfigurieren, dass der Branch eines neu angelegten Repositories automatisch *main* genannt wird.

```
git config --global init.defaultBranch main
```

3.1.3 Eine Datei erstellen

Generell ist ist (natürlich) möglich, jeden Typ von Datei mit *git* zu versionieren. Eine der Stärken von *git* ist allerdings der Vergleich von Versionsständen auf Zeichenebene und deshalb sind Binärformate wie Musik, Bider, aber auch Dokumente der gängigen Office-Produkte nur bedingt geeignet.

Erstelle nun eine einfache Datei mit Hilfe eines Editors (notepad, notepad++, Programmierumgebung, …) mit dem Namen *datei1.txt*, die als Inhalt nur den Inhalt *Version 1* enthält.

Auch hier wieder Version mit dem Terminal:

```
echo "Version 1" > datei1.txt
```

Hinweis

Das Symbol > überschreibt den Inhalt der Datei ohne Warnung, mit >> wird der Text an das Ende der Datei angehängt.

3.1.4 Versionieren – einfachste Form

Dieser Abschnitt ist im Prinzip relativ kurz, solange keine Probleme auftreten:

Nimm die Datei in die Versionierung auf

```
git add datei1.txt
```

und schreibe einen Eintrag in die Projekt-History

```
git commit -m "Erste Datei erstellt"
```

Natürlich kommen die beiden Befehle hier in deutlich zu kurzem Abstand! Der generella Ablauf sieht aber genau so aus:

- Dateien, in denen du Änderungen vorgenommen hast, sammelst du mit *add*
- Haben sich genug Daten angesammelt – mit Sinnzusammenhang – dann erstellst du einen *Commit* mit einer aussagekräftigen Beschreibung.

Hinweise
Wenn du die Beschreibung lieber mit dem Editor machen möchtest, rufts du *nur* `git commit` auf.

In welche Zeitabständen oder Sinnzusammenhängen du Commits erstellst, ist alleine *deine* Entscheidung.
Du solltest aber bedenken, dass ein Rücksprung zu einem älteren Commit zunächst von *allen* Dateien die Vorgängerversion restauriert. Wenn du also zu viel Änderungen am Code vorgenommen hast, die behalten willst, dann wirst du einiges an Handarbeit haben. Deshalb lieber mehr Commits in kürzeren Abständen!

Ändere deine Datei nun ab, indem du eine Zeile ergänzt:

```
echo "Version 2" >> datei1.txt
git add datei1.txt
git commit -m "Erläuterung"
```

Die Historie besitzt nun zwei Punkte, an die du zurückgekehren könntest – wie das geht, lernst du noch.

Hinweis
Du kannst natürlich auch mehrere Dateien in einem Rutsch hinzufügen!

```
git add .  # Alle Dateien im aktuellen Ordner
           # und den Unterordnern

git add --all  # (oder -A) fügt alle Dateien
               # ab der obersten Ebene des
               # Projekts hinzu
```

Wir werden später sehen, was genau beim *add* passiert, z.B. wenn man die gleiche Datei zwischen zwei Commits mehrfach ändert und *addet*.

3.1.5 Versionieren – etwas genauer betrachtet

Aktuell hast du zwei Commits und keine neuen Änderungen an den Dateien. Diesen Zustan solltest du sehen, wenn du den Status abfragst:

```
git status
```

4 Hands On I

Bevor wir beginnen, müssen einmalig einige Konfigurationen vorgenommen werden – einige davon betreffen auch bereits den späteren Einsatz im Netz, wo klar sein muss, welcher Benutzer Änderungen an den Dateien vorgenommen hat[1].

TIPP
Im Terminal kann mit der *Pfeil hoch* Taste durch die letzten Befehle geblättert werden – damit gehen manche Dinge sehr schnell!

4.1 Konfigurieren von git

Öffne die *git*-Bash[2]. Da es sich um globale Einstellungen handelt, muss das nicht unbedingt im Projektordner sein![3].

Aktuelle Einstellungen ansehen

```
# Eingabe
git config --list

# Ausgabe
user.email=
user.name=
     ...
```

Einstellungen anpassen

```
# eigene Phantasiedaten verwenden
git config --global user.name="Susi Sandmann"
git config --global user.email="susi@sandmann.de"
```

4.2 Erste Schritt

Lege nun einen Arbeitsordner für dieses *Hands on* an:

[1] Wir sprechen hier NICHT von einer Absicherung gegen Dummheit oder Bosheit. Sobald Änderungen von papst@vatikan.va auftauchen, wäre ich skeptisch!
[2] bzw. das Terminal deines Systems. Das setze ich in Zukunft stillschweigend voraus.
[3] Sollte dir still die Schrift im Fentser zu klein sein, kannst du sie mit strg + + vergrößern.

```
git init hands_on_1
cd hands_on_1
```

Alternativ kannst du das auch mit mehreren Schritten erledigen:

```
mkdir hands_on_1
cd hands_on_1
git init
```

Es erscheint eine Meldung, die je nach System etwas anders aussehen kann:

```
Hinweis: Als Name für den initialen Branch wurde 'master' benutzt.
Hinweis: Dieser Standard-Branchname kann sich ändern.
Hinweis: Um den Namen des initialen Branches zu konfigurieren,
Hinweis: der in allen neuen Repositories verwendet werden soll und
Hinweis: um diese Warnung zu unterdrücken, führen Sie aus:
Hinweis:
Hinweis:    git config --global init.defaultBranch <Name>
Hinweis:
Hinweis: Häufig gewählte Namen statt 'master' sind 'main', 'trunk'
Hinweis: und 'development'. Der gerade erstellte Branch kann
Hinweis: mit diesem Befehl umbenannt werden:
Hinweis:
Hinweis:    git branch -m <Name>
Leeres Git-Repository in /tmp/hands_on_1/.git/ initialisiert
```

Es hat sich eingebürgert[4], den Arbeitszweig von *master* auf *main* umzubenennen:

```
git branch -m main
```

Den Erfolg kannst du schnell überprüfen:

```
git status
```

Es sollte folgender Text (eventuell englisch) erscheinen:

```
Auf Branch main

Noch keine Commits

nichts zu committen (erstellen/kopieren Sie Dateien und benutzen
Sie "git add" zum Versionieren)
```

Hinweis
Du kannst *git* auch so konfigurieren, dass der Branch eines neu angelegten Repositories automatisch *main* genannt wird.

```
git config --global init.defaultBranch main
```

[4] Bedingt durch gesellschaftspolitische Strömungen, die *Master* mit Sklaverei gleichsetzen, aber Eine Master-Arbeit hier nicht als Problem sehen.

Erstelle nun eine erste Datei mit dem Namen *datei1.txt*.
Wenn du das nicht über die *git*-Bash machen willst, dann achte darauf, dass du eine reine Textdatei erstellst – also nicht mit Word o.ä. sondern mit einem reinen Editor wie *notepad*, *notepad++* oder einer Entwicklungsumgebung.

Schneller geht das aber mit *git*-Bash[5].

```
echo "Version 1" > datei1.txt
```

Diese Datei nimmst du dann in die Versionsverwaltung auf

```
git add datei1.txt
```

und erstellst einen Commit

```
git commit -m "Erste Datei erstellt"
```

Durch die Kombination aus *add* und *commit* hast du einen *Meilenstein* in der Projekthistorie erstellt. Damit kennst bereits einen typischen Ablauf von *git*:

Du sammelst Dateien mit *add*, in denen du Änderungen vorgenommen hat und schreibst diese Änderungen mit *commit* in die Projekthistorie. Wie oft, mit wie vielen Dateien in welchen Zeitabständen … das ist ziemlich alleine *deine* Entscheidung.

Generell gilt allerdings: „Lieber mehr Commits, als zu wenige!“

Hinweis
Ein Commit ist ein Projektstand, an den du zurückkehren kannst, wenn etwas grob missglückt ist. Vergeht zu viel Zeit (=viel Code) zwischen zwei Commits, so entstehen eventuell größere Verluste bei einem Rücksprung zum alten Stand.
Später im Workshop lernst du auch, wie du Commits nachträglich zusammenfassen kannst (=squash) um deine Entwicklungsschritte übersichtlich zu halten.

Ändere deine Datei nun ab, indem du eine Zeile ergänzt:

```
echo "Version 2" >> datei1.txt
git add datei1.txt
git commit -m "Erläuterung"
```

Die Historie besitzt nun zwei Punkte, an die du zurückgekehren könntest – wie das geht, lernst du noch.

Natürlich ist das *adden* nicht auf eine einzige Datei beschränkt. Liegen mehrere geänderte Dateien vor, so müssen sie nicht einzeln hinzugefügt werden. Es gibt dafür einen *Sammelbefehl*, der den gesamten Ordner mit allen Unterordnern hinzufügt:

```
git add .
```

Das *adden* ist generell unabhängig vom *commit*. Es ist sogar üblich, immer erst einige Dateien anzusammeln, bevor du einen Commit machst.
Es ist auch völlig unproblematisch eine Datei zwischen zwei Commits mehrfach in der aktuellen Version mit *add* für den Commit zu registrieren.

[5] Das Zeichen > überschreibt alle Inhalte einer Datei, >> hängt die Daten hinten an.

Es kann ab und zu passieren, dass du Änderungen an Dateien vornimmst, die zu verschiedenen Problemen gehören. Es wäre dann etwas unübersichtlich, wenn
diese Dateien dann im gleichen Commit festgeschrieben werden. In einem solchen Fall solltest du bei *add* genau die Dateinamen angeben.

Zusätzlich kann es natürlich auch sein, dass du bestimmte Dateien überhaupt nicht in die Versionsverwaltung aufnehmen willst. Das ist zum Beispiel dann der Fall, falls sich Kennwörter und Zugangsdaten in eine Datei befinden. Wenn du die Namen dieser Datei(en) in eine Datei mit dem Namen `.gitignore` einträgst, werden sie beim *add* ignoriert.

```
# Format der Datei
*.log
*.tmp
zugangsdaten.txt
```

Mit diesen Befehlen ist bereits ein einfacher lokaler Entwicklungsprozess abbildbar – allerdings gibt es noch viele weitere Feinheiten und Varianten.

Hinweis
Es gibt sogar die Möglichkeit, dass man nur Teile der Änderungen in einer Datei in einen Commit und die restlichen Änderungen in einen anderen Commit übernimmt. Das sind aber sehr spezielle Varianten, die wir nicht behandeln!

4.3 Ein genauerer Blick auf Git

Steht ein Ordner unter der Verwaltung von *git*, so kann man von drei *logischen* Bereiche sprechen:

- *Working-Directory* (Arbeitsordner)
- Stage (auch Index oder Cache genannt)
- Repository[6]

Das *Working-Directory* enthält die Dateien an denen gearbeitet wird. Er entspricht auf den ersten Blick genau einem normalen Ordner für die Projektentwicklung. Erst durch die Initialisierung durch *git* entsteht dann ein weiterer, versteckter Ordner mit dem Namen *.git*. Dieser Ordner bildet gleichzeitig den *Stage* und das *Repository*[7].

Logische Bereiche bedeutet in diesem Zusammenhang, dass diese beiden Bereiche von *git* verwaltet werden und vom unbedarften Benutzer nicht einfach unterschieden werden können. In einem späteren Abschnitt werden wir einen genaueren Blick in diesen Ordner werfen.

Betrachten wir zunächst einen Entwicklungszyklus im Detail. Dabei wirst du im Wesentlichen die gleichen Befehle wie vorher verwenden, allerdings untersuchen wir einige Varianten, die für das Verständnis wichtig sind.

Du bist immer noch im aktuellen Ordner *hands_on_1*. Er sollte akutell eine Datei und zwei Commits enthalten.
Das kannst du mit folgendem Befehl überprüfen:

[6] Je nach Quelle und Kontext etwas inkonsistent verwendet
[7] Hier siehst du den inkonsistenten Gebrauch von „Repository“. Einerseits wird der gesamte Projektorder als Repository bezeichnet, sobald er von *git* verwaltet wird, andererseits befindet sich das eigentliche Repository im Ordner *.git*.

```
git log
```

liefert

```
commit 6db4f3383512a7f3aaaf3b050334d7fd2c474688 (HEAD -> main)
Author: Wolfgang <hoeferwolf@t-online.de>
Date:   Thu Nov 13 10:46:50 2025 +0100

    Erläuterung

commit 430149c83906005ae2c53c8eaf9ce230e3456ad0
Author: Wolfgang <hoeferwolf@t-online.de>
Date:   Thu Nov 13 10:46:50 2025 +0100

    Erste Datei erstellt
```

Ob noch Änderungen offen sind,zeigt dir:

```
# Eingabe
git status

# Ausgabe (Zeilenumbrüche geändert)
Auf Branch main
nichts zu committen,
Arbeitsverzeichnis unverändert
```

Das bedeutet, dass du aktuell einen *sauberen* Zustand deines Arbeitsordners ohne Änderungen vorliegen hast.

Erstelle nun eine neue *datei2.txt* und füge die Zeile *Version 1* ein. In der Datei *datei1.txt* kommt die Zeile *Version 3* dazu:

```
# Eingabe (Lasse ich zukünftig weg)
echo "Version 3" >> datei1.txt
echo "Version 1" >> datei2.txt

# Status anzeigen lassen
git status

# Ausgabe
Auf Branch main
Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)

  (benutzen Sie "git restore <Datei>...",
   um die Änderungen im Arbeitsverzeichnis zu verwerfen)

    geändert:       datei1.txt

Unversionierte Dateien:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)
```

```
    datei2.txt

keine Änderungen zum Commit vorgemerkt
(benutzen Sie "git add" und/oder "git commit -a")
```

Es wird dir angezeigt, dass Änderungen im *Working-Directory* vorliegen.
Du siehst auch, dass *git* die *datei1.txt* bereits kennt (=geändert) und dass *datei2.txt* noch nicht in die
Versionsverwaltung aufgenommen wurde (=Unversioniert).

Hinweis
In Bezug auf *add* muss man zwei Dinge im Hinterkopf behalten. 1. Die Abkürzung `git commit -a` führt zwar im Hintergrund ein *add* aus, aber nur für bereits versionierte Dateien – *datei2.txt* würde also ignoeriert.
2. Der Befehl `git add .` findet nur Dateien im aktuellen Ordner und den Unterordnern. Sollen auch Dateien in übergeordneten oder *parallelen* Ordnern berücksichtigt werden, so brauchst du `git add --all`.

Nun werden die aktuelle Stände *auf den Stage/in den Stage* übertragen:

```
# Deien aufnehmen
git add .

# Status abzeigen
git status

# Ausgabe
Auf Branch main
Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git restore --staged <Datei>..."
   zum Entfernen aus der Staging-Area)

    geändert:       datei1.txt
    neue Datei:     datei2.txt
```

Diese Dateien sind nun zum Commit vorgemerkt, also zum festen Eintrag in die Projekthistorie – führst du aber **nicht aus**!.

Als Demonstration fügen wir beiden Dateien eine weitere Zeile hinzu und fragen wieder den Status ab:

```
echo "Weitere Zeile" >> datei1.txt
echo "Weitere Zeile" >> datei2.txt
git status

# Ausgabe ( "----"" sind Anmerkungen )
Auf Branch main
Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git restore --staged <Datei>..."
   zum Entfernen aus der Staging-Area)

    geändert:       datei1.txt
```

```
      neue Datei:     datei2.txt

--------------- Ende Stage ------------
--------------- Anfang Workdir --------

Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...", um die Änderungen
   zum Commit vorzumerken)

  (benutzen Sie "git restore <Datei>...",
   um die Änderungen im Arbeitsverzeichnis zu verwerfen)

    geändert:       datei1.txt
    geändert:       datei2.txt
```

Beide Dateien liegen also in verschiedenen Versionen im Stage und im *Working-Directory* vor.
Ein Commit transferiert die Dateien aus dem Stage ins Repository, also in die Projekthistorie. Das führt dann zu folgender Situation:

```
git commit -m "nur Stage"
```

Ein `git status` zeigt dir, dass die oberen beiden Dateiversionen aus dem Stage verschwunden sind (sind jetzt Teil des Commits) und die aktuellen Änderungen nach wie vor im Arbeitsverzeichnis vorhanden sind, weil für sie noch kein *add* stattgefunden hat.

```
git Status

# Ausgabe
Auf Branch main
Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...", um die Änderungen zum Commit vorzumerken)
  (benutzen Sie "git restore <Datei>...", um die Änderungen im Arbeitsverzeichnis zu verwerf
    geändert:       datei1.txt
    geändert:       datei2.txt

keine Änderungen zum Commit vorgemerkt (benutzen Sie "git add" und/oder "git commit -a")

# Zum Stage hinzufügen
git add

git status

# Ausgabe

Auf Branch main
Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git restore --staged <Datei>..." zum Entfernen aus der Staging-Area)
    geändert:       datei1.txt
    geändert:       datei2.txt
```

Hinweis
Wenn ich hier von *transferieren* spreche, so führt das zu einer falschen Vorstellung. In Wirklichkeit werden im *.git*-Ordner nur einige Verweise (Links, Zeiger) geändert – wobei das auch wieder nicht die ganze Wahrheit ist!

4.3.1 Etwas mehr Wahrheit

Dieser Abschnitt ist nur als Einblick für Interessierte gedacht!

Ganz am Anfang muss man wissen, dass es 4 Git-Objekte gibt:

1. blob
2. tree
3. commit
4. tag

blob
„Binary large Objects“ sind reine Dateiinhalte *ohne* Dateiname oder Metadaten.

tree
Ein Tree ist ein (namentliches) Abbild des Arbeitsverzeichnisses. Er enthält aber nur die Namen, Typ und Hash der Dateien, Unterordner, also der Blobs und trees, die an anderer Stelle gespeichert sind. Ein Tree ist klein, da er keine Dateiinhalte enthält.

commit
Zeigt auf den Tree, der zum Zeitpunkt des Commits aktuell war. Er enthält noch weitere Metadaten.

tag
Zusatznamen für ausgewählte Commits (Releases)

Sehen wir uns das genauer an

Erstelle dir ein neues Repository in einem anderen Ordner

```
cd ..  # Eine Ebene höher, raus aus dem Arbeitsordner
git init demo
cd demo
git branch -m main
```

Wenn du nun die Verzeichnisstruktur des Ordners *.git* genauer untersuchst, wirst du in etwa dies hier sehen, weil noch keine Dateien im *Working-Directory* sind:

```
.git
├── branches
├── config
├── description
├── HEAD
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── prepare-commit-msg.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── push-to-checkout.sample
│   ├── sendemail-validate.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

```
.git/
  branches
  config
  description
  HEAD
  hooks
      applypatch-msg.sample
      commit-msg.sample
      fsmonitor-watchman.sample
      post-update.sample
      pre-applypatch.sample
      pre-commit.sample
      pre-merge-commit.sample
      prepare-commit-msg.sample
      pre-push.sample
      pre-rebase.sample
```

```
        pre-receive.sample
        update.sample
    info
        exclude
    objects
        info
        pack
    refs
         heads
         tags
```

Lege nun im leeren *Working-Directory* eine Datei an[8]

```
echo "Etwas HTML" >> homepage.html
git add homepage.html
```

Du siehst erste Änderungen im *.git*-Ordner. Im Bild siehst du aus Platzgründen nur den relevanten Ausschnitt. Bei dir kann der Hash einen anderen Wert besitzen!

```
│   └── update.sample
├── index
├── info
│   └── exclude
├── objects
│   ├── ce
│   │   └── 1332fcf1795166c3c5cd0216b5e7dc2dce7998
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

```
.
.
.
    index            <<< neu
    info
        exclude
    objects
        ce          <<< neu, bei dir evtl anders!
            1332fcf1795166c3c5cd0216b5e7dc2dce7998
        info
        pack
```

Die Datei *index* stellt den Stage dar. Diese Datei kannst du nicht direkt lesen. Das geht nur mit

```
git ls-files --stage
```

[8] Ich habe absichtlich nicht index.html verwendet, weil „index“ aktuell anders belegt ist.

Den Hash aus dieser Ausgabe[9] findet du oben in der Ordnerstruktur wieder.

```
100644 ce1332fcf1795166c3c5cd0216b5e7dc2dce7998 0   homepage.htm
```

Allerdings wird mit den ersten beiden Zeichen (ce) ein Ordner erstellt und in diesem Ordner eine Datei, deren Name aus den restlichen Zeichen besteht. Zusätzlich findet man den Dateityp (100644), den Status (Das ist komplexer! Hier 0) und den Dateinamen.

Den Hash kannst du dir übrigens auch so anzeigen lassen:

```
git hash-object homepage.htm
```

Aber gehen wir einen Schritt weiter und committen die Datei:

```
git commit -m "homepage.html added"
```

Nun sieht die Ordnerstruktur verändert aus – die Hooks habe ich in der Darstellung aus Platzgründen entfernt!

```
│   └── update.sample
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
│           └── main
├── objects
│   ├── 94
│   │   └── eed25c4ef6cfa1384df66f28308c09dc9bf28d
│   ├── b7
│   │   └── 510dd34252cb3e42bfc82f043d9f42aa00b2f5
│   ├── ce
│   │   └── 1332fcf1795166c3c5cd0216b5e7dc2dce7998
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── main
    └── tags
```

```
.git/
  branches
  COMMIT_EDITMSG
  config
  description
  HEAD
```

[9] In einem richtigen großen Repository erscheint hier eine sehr umfangreiche Liste aller Dateien!

```
hooks
  ....
index
info
    exclude
logs
    HEAD
    refs
        heads
            main
objects
    3b
        028e4d0230fb8f3553f0f7579e68e6c4e27d3f
    94
        eed25c4ef6cfa1384df66f28308c09dc9bf28d
    ce
        1332fcf1795166c3c5cd0216b5e7dc2dce7998
    info
    pack
refs
    heads
        main
    tags
```

Ab hier bekommst du sicher andere, eigene Hash-Werte! Du musst in den nachfolgenden Befehlen also deine eigenen Werte verwenden.

Wir konzentrieren uns auf die *objects* und das *log*.

Mit `git log` erfährst du etwas über den erfolgten Commit – zum Beispiel seinen Hash:

```
commit b7510dd34252cb3e42bfc82f043d9f42aa00b2f5 (HEAD -> main)
Author: wolfgang <hoeferwolf@t-online.de>
Date:   Sun Jan 26 20:20:10 2025 +0100

    homepage.html added
```

Etwas kompakter wird die Darstellung mit `git log --oneline`.

Der Hash b751… scheint also ein Commit zu sein. Das kannst du mit `git cat-file -t b7510dd34252cb3e42bfc82f043d9f42aa00b2f5` auch noch einmal überprüfen – steht aber bereits in `git log`.

Änderst du den Parameter von *-t* auf *-p* (=pretty print), so bekommst du weitere Details:

```
# Eingabe
git cat-file -p b7510dd34252cb3e42bfc82f043d9f42aa00b2f5

# Ausgabe
tree 94eed25c4ef6cfa1384df66f28308c09dc9bf28d   << wichtig!
author wolfgang <hoeferwolf@t-online.de> 1737919210 +0100
committer wolfgang <hoeferwolf@t-online.de> 1737919210 +0100

homepage.html added
```

Am Commit verweist also auf den *tree* mit dem Hash 94ee…
Da wir aktuell nur eine Datei haben, ist die Ausgabe des nächsten Befehls sehr kurz, der den tree ausliest:

```
# Eingabe
git cat-file -p 94eed25c4ef6cfa1384df66f28308c09dc9bf28d

# Ausgabe - normal deutlich länger!
100644 blob ce1332fcf1795166c3c5cd0216b5e7dc2dce7998    homepage.html
```

Abschließend sehen wir uns den Inhalt des Blob an

```
# Eingabe
git cat-file -p ce1332fcf1795166c3c5cd0216b5e7dc2dce7998

# Ausgabe
Etwas HTML
```

Du bist jetzt über mehrere Stufen zum Inhalt der Datei gelangt. Eine kleine Stufe haben wir aber ausgelassen:

```
git cat-file -t ce1332fcf1795166c3c5cd0216b5e7dc2dce7998
```

liefert die Information `blob` (Binary large object).

Hier nochmal die Zusammenfassung der Funktionsweise:

- Der Commit zeigt auf einen Tree
- Im Tree stehen Dateinamen mit den Hashes
  Das sind nur die Namen und nicht die Dateien!
- Der Hash gehört zu einem *blob* (oder tree),
  also den reinen *Roh*daten der Datei **ohne** Dateinamen

Git verwendet das, um von verschiedenen Stellen aus auf die gleichen Rohdaten verweisen zu können, ohne Speicher zu verbrauchen. Ändern sich Dateien zwischen zwei Commits nicht (Hash bleibt gleich), so werden sie nicht neu gespeichert. Der neue Commit zeigt auf einen neuen Tree in dem wieder die gleichen Hashes mit den gleichen Dateinamen stehen.

Tiefer will ich an dieser Stelle in dieses Thema nicht einsteigen!

4.4 Fehlerbehebung

Ein Versionsverwaltungsprogramm hat unter anderem die Aufgabe, ein Projekt in gewissem Umfang *revisionssicher* zu protokollieren. Aus diesem Grund müssen bestimmte *Fehler* oder *dokumentierte Probleme* in der History erhalten bleiben, auch wenn sie später behoben wurden.

Allerdings treten auch bei der Verwendung von *git* Fehler auf – im Sinne von Bedienfehlern, zu frühem Committen, …
Derartige Fehler müssen repariert werden, bevor sie endgültig in der Historie festgeschrieben werden.

Relativ unproblematisch sind *Reparaturen* bei lokaler Arbeit, weil keine anderen Team-Mitglieder betroffen sind. Nachfolgend besprechen wir einige Szenarien mit kleinen

Beispielen. Aktuell ist das Arbeiten mit verschiedenen Entwicklungszweigen noch nicht Thema!

4.4.1 Kleiner Fehler 1

Der Fehler ist ganz frisch …

```
git init beispiel_1
cd beispiel_1
git branch -m main

# alles funktioniert
echo "Alles gut" > datei.txt
git add .
git commit -m "alles gut"

# Hier kommt der Fehler
echo "Mist gebaut" >> datei.txt

###### Alte Version wieder herstellen
git restore datei.txt
```

4.4.2 Kleiner Fehler 2

Der Fehler wurde bereits ge-add-et.

```
########### IRRWEG !!!!!!!!! #########
git init beispiel_2
cd beispiel_2
git branch -m main

# alles funktioniert
echo "Alles gut" > datei.txt
git add .
git commit -m "alles gut"

# Hier kommt der Fehler
echo "Mist gebaut" >> datei.txt
git add .

#####################
###### Das geht jetzt nicht, weil Stage = Workdir
#####################
git restore datei.txt
```

Du musst nun mit `git log` den Hash vom letzten Commit ermitteln und dann geht

```
git restore --source=<HASH> datei.txt
```

4.4.3 Größerer Fehler 1

Viele Fehler nach letztem Commit

```
git init beispiel_3
cd beispiel_3
git branch -m main

# alles funktioniert
echo "Alles gut datei1" > datei1.txt
echo "Alles gut datei2" > datei2.txt
# ... viele Dateien
git add .
git commit -m "alles gut"

# Hier kommen DIE Fehler
echo "Mist gebaut" >> datei1.txt
echo "Mist gebaut" >> datei2.txt
# ... viele Fehler in vielen Dateien

# Rücksetzen des kompletten Projekts auf den letzten Commit
git reset --hard HEAD
```

Sind viele Dateien *verkorkst*, so bietet sich ein komplettes Rücksetzen auf einen alten Stand an. Allerdings gibt es da einige Varianten (siehe Tabelle unten).

Wird der Befehl ohne `--hard` ausgeführt, so wird nur der Stage auf den Stand des alten Commit gesetzt, nicht aber das *Working-Directory*. Aber eigentlich verlassen wir hier schon die einfachen Fälle. Ich persönlich verliere bei so vielen Varianten bald den Überblick und von einem *routinierten Handling* kann keine Rede sein!

Befehl (Beispiel)	Wirkung	Warnung / Hinweis
`git reset --hard HEAD`	Setzt Arbeitsverzeichnis und Staging auf `HEAD` zurück; alle uncommitted Änderungen werden gelöscht.	Unwiderruflich, wenn Änderungen nicht anderweitig gesichert sind.
`git reset --hard HEAD~1`	Branch-Zeiger bewegt sich ein Commit zurück; Arbeitsverzeichnis entspricht dem vorletzten Commit.	Vorsichtig bei bereits gepushten Commits — Remote bleibt davon unberührt.
`git reset --soft HEAD~1`	Entfernt letzten Commit, macht die Änderungen als gestaged verfügbar (Index enthält sie).	Gut zum Kombinieren/Neu-formulieren von Commits.
`git reset --mixed HEAD~1` (oder `git reset HEAD~1`)	Entfernt Commit; Änderungen bleiben im Working Directory, aber sind nicht gestaged.	Standardmodus (`--mixed`) wenn kein Flag angegeben.
`git reset HEAD`	Entfernt Dateien aus dem Index; Änderungen bleiben im Arbeitsverzeichnis.	Nützlich, um versehentliches `git add` rückgängig zu machen.

Befehl (Beispiel)	Wirkung	Warnung / Hinweis
`git reset --hard <commit-hash>`	Setzt Branch auf den angegebenen Commit; Arbeitsverzeichnis wird entsprechend angepasst.	`<commit-hash>` vorher mit `git log`/`git rev-parse` prüfen.
`git reset --hard HEAD~1` + `git push --force`	Lokaler und Remote-Branch zeigen auf den älteren Commit (push überschreibt Remote).	**Gefährlich**: ändert history auf Remote — Absprache mit Team erforderlich.

4.4.4 Da war doch mal was

Machnmal – gerade bei Dokumentationen oder anderen Texten – hätte man gerne die Version zurück, die man vor einigen Tagen gelöscht hat. Bei *git* hast du Zugriff auf alle alten Versionen! Allerdings ist oft die Frage, in welchem Commit sich die Version versteckt. Hier wird es nötig, sich rückwärts durch die ensprechenden Commits zu arbeiten. Dabei gibt es allerdings ein Besonderheit, die du kennen musst. Nachfolgende Script erzeugt das passende Szenario:

```
git init suche
cd suche
git branch -m main

for i in {1..5};do
 echo "Version $i" > datei.txt
 git add .
 git commit -m "Version $i"
done

echo "gewünschte Version" > datei.txt
git add .
git commit -m "Version 6"

for i in {7..10};do
 echo "Version $i" > datei.txt
 git add .
 git commit -m "Version $i"
done
```

Du weißt aus dem Script, dass Version 6 die Datei in der gewünschten Form enthält. Im Ernstfall weißt du das wahrscheinlich nicht. Du beginnst also mit der Suche:

```
git log --oneline

# Ausgabe
b55b329 (HEAD -> main) Version 10
0149855 Version 9
a4bc57f Version 8
95a37c9 Version 7
d370ffb Version 6
25fd886 Version 5
```

```
c745eda Version 4
4a64eef Version 3
68a4d14 Version 2
461eee6 Version 1
```

Am Besten kopierst du dir die Ausgabe in eine Datei, damit du immer Zugriff auf die Hashwerte der Commits hast. Nimm nun als Beispiel einen Sprung zu Version 7.

```
git checkout 95a37c9
```

Nun kommt eine große Warnung, die dich darüber informiert, dass du gerade außen an der Versionsverwaltung entlang krabbelst. Da du noch nicht mit Branches arbeitest, solltest du aktuell keinerlei Änderungen an den Dateien vornehmen! In die Dateien hinein sehen ist keine Änderung – das ist also ok. Du kannst auch Teile der Inhalte wegkopieren um sie später weiterverwenden zu können.

```
Hinweis: Wechsle zu '95a3'.

Sie befinden sich im Zustand eines 'losgelösten HEAD'. Sie können sich
umschauen, experimentelle Änderungen vornehmen und diese committen, und
Sie können alle möglichen Commits, die Sie in diesem Zustand machen,
ohne Auswirkungen auf irgendeinen Branch verwerfen, indem Sie zu einem
anderen Branch wechseln.

Wenn Sie einen neuen Branch erstellen möchten, um Ihre erstellten Commits
zu behalten, können Sie das (jetzt oder später) durch Nutzung von
'switch' mit der Option -c tun. Beispiel:

  git switch -c <neuer-Branchname>

Oder um diese Operation rückgängig zu machen:
  git switch -

Sie können diesen Hinweis ausschalten, indem Sie die Konfigurationsvariable
'advice.detachedHead' auf 'false' setzen.

HEAD ist jetzt bei 95a37c9 Version 7
```

4.4.5 Es ging nur wenig schief

Du hast nach dem letzten Commit an einer Datei weiter gearbeitet, bist mit dem Ergebnis aber nicht zufrieden. Du möchtest die letzte Version der Datei wieder herstellen:

```
git init sz1
cd sz1
git branch -m main

echo "Alles gut" > datei.txt
git add .
git commit -m "alles gut"
```

```
echo "Mist gebaut" >> datei.txt

###### Zurück rollen
git status
git restore datei.txt
```

Bereits versionierte Dateien mit Fehlern können durch `git restore <dateiname>` wieder mit der Version aus dem Stage überschrieben werden.

Vorher habe ich geschrieben, dass bei einem Commit die Dateien aus dem Stage in die Projekthistorie übertragen und aus dem Stage entfernt werden. Intern funktioniert das allerdings etwas anders. Die Datei *index* enthält die Namen und Hashes des aktuellen Commits (tree). Mit einem *add* werden Dateinamen ergänzt. Beim Commit wird die Datei nicht geleert, die Hashes bleiben also verfügbar. Dass `git status` diese Dateien nicht mehr anzeigt, liegt einfach daran, dass der Befehl eigentlich nur ein *diff* zwischen *index* und aktuellem *tree* ausführt.

Dass der Stage wirklich nicht leer ist, zeigt folgendes Beispiel, das eine Reparatur am *Working-Directory* zeigt.

```
echo "text" > datei.txt
git add datei.txt     # danach im Stage
git commit -m "gesichert"

git status   # Stage anscheinend leer

echo "falscher Text" >> datei.txt   #  Fehler im Arbeitsverzeichnis
git restore datei.txt              # repariert mit Datei aus Stage
```

Hinweis
Im Netz findet man oft die Variante mit `checkout`, die früher üblich war. `restore` wurde eingeführt um eindeutigere Befehle zu schaffen.

Mit dem `restore`-Befehl kann man auch eine Datei aus einem beliebigen Commit restaurieren, von dem man den Hash kennt:

```
git restore --source=<HASH> <datei>
```

Die Fehlerbehebung kann auch das Löschen ganzer Entwicklungs- oder Testzweige (Branches) bedeutet, was wir später noch besprechen.

- Manchmal soll auch nur die Version gelöscht werden, die irrtümlich auf den Stage geschoben wurde. Dabei darf das *Working-Directory* nicht verändert werden. Das erreichst du mit `git restore <dateiname> --staged`.
  Das braucht man eigentlich nur, wenn diese Datei doch nicht im nächsten Commit enthalten sein soll.

4.4.6 Was ist mit Revert?

Ja – `revert` gibt es und es spielt bei der Arbeit im Team auch eine große Rolle. Bei der lokalen Arbeit wirst du diesen Befehl praktisch nie brauchen – `reset` und `checkout` sind

hier meist einfacher und schneller. Du wirst im Abschnitt über die Teamarbeit aber noch mehr darüber hören!

Der Befehl `revert` wird meistens in einem Atemzug mit `reset` genannt, hat aber einen deutlich anderen Anwendungsbereich. Hierfür müssen wir etwas auf die Arbeit mit entfernten Repositories vorgreifen

Szenario

Du hast einen Commit gemacht und programmierst weiter. Du erstellst einige neue Dateien und erweiterst bestehenden Code in anderen Dateien. Nach deinem nächsten Commit erkennst du, dass du durch einen Denkfehler in einigen Dateien viel Code kaputt gemacht hast und es wäre zu viel Aufwand, die ganzen Fehler per Hand rückgängig zu machen. Nun kannst du `reset` verwenden (aber es gibt natürlich auch andere Möglichkeiten). Je nach Parameter erzielst du nun unterschiedliche Ergebnisse:

- `git reset HEAD`

  Der letzte Commit wird verworfen, der aktuelle Bearbeitungsstand (mit den Fehlern) ist immer noch vorhanden. Das ist sinnvoll, wenn du einige Dateien behalten und andere zurücksetzen willst.

- `git reset HEAD 1`

  Alternative Schreibweise, die aber auch auf frühere Commits anwendbar ist `~2, ~3`.

- `git reset --hard`

  Die Dateien werden auf den Zustand nach dem Commit gesetzt. Alle Fehler sind weg – sinnvoller neuer Code aber auch!

Da in den ersten beiden Varianten die Fehler noch alle vorhanden sind, musst diese noch ausbessern. Willst du das durch Rücksetzen (reset) der Dateien erreichen, brauchst du nun

```
git checkout <HASH> -- <dateiname>
```

Dieser Weg ist völlig ok, solange du nur lokal arbeitest und den Commit non nicht in ein entferntes Repository übertragen hast. Durch diesen Reset verlieren dein lokales und dein entferntes Repository die Synchronisation. Ein neuer Commit mit Push nach re

Es gibt noch (mindestens) einen weiteren Befehl, um Fehler in der Commit-History auszubügeln: `git revert <HASH>`.

Von der Idee her geht es darum, dass Änderungen in einem früheren Commit wieder zurückgesetzt werden. Dafür wird zuerst analysiert, welche Änderungen der *schlechte* Commit eingeführt hat (das wird mit `diff` gemacht). Dann werden diese Änderungen gewissermaßen *negiert* und ein Patch erstellt, der auf den aktuellen Stand angewendet wird. Das Ergebnis ist ein neuer Commit, der diesen Prozess dokumentiert.

Betrachte folgendes (extrem einfaches) Beispiel. Ein Programmierer schreibt ein Programm zum Berechnen des Body-Mass-Index (BMI). Er committet extrem kurzschrittig und hat in einem bestimmten Commit einen Fehler produziert, das er den Variablentyp bei der Parameterübergabe einer Funktion falsch wählt – er erkennt später, dass es *double* sein sollte.

```
public static void ausgabe(String bmi){
        System.out.print("Dein BMI beträt:");
```

```
        System.out.print(bmi);
    }
```

Im nachfolgenden Repository ist die Entwicklung nachvollziehbar:

```
git clone git@gitlab.com:dillingen/bmi.git
```

Da es nur um eine einzige Datei geht, kannst du direkt in die einzelnen Versionen hinein sehen[10], wenn di den Hash des Commits kennst:

```
git log --oneline

# Ausgabe
c83ae97 (HEAD -> main, origin/main)
      Hier wird der Fehler String vs. double erkannt
5eb974e Berechnung BMI
834e608 Eingabe Gewicht
207e388 Ausgabe verfeinert - DAS IST DER FEHLER
67b0a17 Eingabe der Körpergröße
33feb0c Mini - Ausgabe
8a6be40 Initial2
630a635 Initial
daef3c8 Start mit Boilerplate
```

Du rufst auf:

```
git show 33feb0c:./bmi/src/Main.java

# ausgabe
public class Main {
    public static void main(String[] args) {

    }

    public static void ausgabe(){
        System.out.print("Dein BMI beträt:");
        System.out.print(42); // später ausbauen
    }
}
```

Beim Commit 207e388 wird der Fehler in den Code eingebaut und bis zu Commit c83ae97 nicht erkannt. Nun kannst du mit

```
git revert 207e388
```

die alte Version zurückbekommen. Das heißt aber *nicht*, dass der falsche Commit verschwindet! Es wird die aktuelle Version durch die damalige Version überschrieben und sofort ein neuer Commit gemacht, um das zu dokumentieren.

[10] bei vielen Dateien ist das unpraktisch!

5 Hands on II

Du wirst in diesem Abschnitt einige Dinge als redundant empfinden, weil sie auch bereits im ersten *Hands On* verwendet wurden. Sieh es einfach als Übung und da du dieses PDF ohnehin nicht ausdrucken wirst, ist die minimal erhöhte Seitenzahl auch kein Problem.

Dieses Szenario ist etwas komplexer, da wir mit mehreren Dateien arbeiten und auch parallele Ideen verfolgen wollen (branching).
Da richtiges Programmieren zu lange dauert, erstellen wir in diesem Projekt ein kleines Buches mit Kurzgeschichten. Die Geschichten wurden von einer KI generiert um den Arbeitsaufwand zu minimieren.

Die Einzelteile der Geschichten befinden sich in einem *git*-Repository und müssen zunächst auf den lokalen Computer geclont werden. Das ist ein notwendiger Vorgriff auf den Abschnitt der Teamarbeit, den du hier einfach mal ausführst. Es handelt sich um ein öffentliches Repository, auf das auch ohne Benutzerkennung zugegriffen werden darf – aber eben nur lesend. Aus diesem Grund wird zum Clonen auch *https* und nicht *ssh* verwendet. Öffne die *git*-Bash (oder ein Terminal) und gib ein:

```
mkdir fortbildung
cd fortbildung
git clone https://TODO    storyvorlagen # Ziel: vorlage.git
cd vorlage
```

Es entsteht ein Ordner *storyvorlagen*, in dem die verschiedenen Stände des Projektes enthalten sind.

5.1 Labor

Es dürfte am einfachsten sein, wenn du zwei Terminals öffnest:

- Eines für den Vorlagenordner
- Eines für deinen Arbeitsordner

Du wirst ständig zwischen diesen Repositories wechseln müssen und da sind einzelne Fenster optimal (Wechsel z.B. mit alt + tab).

Um der Erklärungen besser folgen zu können, wäre es sinnvoll, wenn bei jedem Teilnehmer exakt die gleichen Hashwerte wie hier in der Anleitung entstehen würden. Leider ist das aber nicht möglich, weil die Hashwerte nicht reproduzierbar sind – zumindest auf meinem Rechner! Aus diesem Grund wurden im Vorlagenordner die Commits *getagged*, so dass du über ihre Namen auf sie zugreifen kannst – wie das geht, siehst du gleich!

Erstellen eines Repositories

Öffne im Fortbildungsordner eine *git*-Bash und erstelle ein Repository für das Buch.

```
# Eingaben
cd
cd fortbildung
git init buch
cd buch
```

Je nach Version von *git* kommt hier eine längere Ausgabe … oder auch nicht. Falls in der Ausgabe der Branch *master* erscheint, bitte umbenennen mit `git branch -m main`

Im Ordner *fortbildungen* sollten nun also die Repositories *storyvorlagen* und *buch* vorliegen.

Erste Datei anlegen

Wechsle in das Fenster mit dem Vorlagenordner und Checke *Step_1* aus. *Auschecken* wird später noch genauer erklärt!

```
git checkout Step_1
```

Kopiere die Datei *story1.md* in den Ordner *buch*.

Sie enthält (absichtlich) nur einen Teil der Geschichte! Das Kopieren kann auch gerne mit der Maus im Explorer stattfinden. Wer allerdings nicht ständig zwischen Tastatur und Maus wechseln möchte, der kann auch folgenden Befehl verwenden:

```
# aktueller Ort ist der vorlagen-Ordner
cp story1.md  ../buch/
```

Wechsle in das Fortbildungsfenster

```
# Eingabe
git status

# Ausgabe
Auf Branch main                                                    ①

Noch keine Commits

Unversionierte Dateien:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)
    story1.md                                                      ②

nichts zum Commit vorgemerkt, aber es gibt unversionierte Dateien
(benutzen Sie "git add" zum Versionieren)                          ③
```

① Geänderter Name ist ok.
② Diese Datei ist betroffen, hier kann eine längere Liste stehen.
③ Empfohlener Befehl `git add story1.md`

Übertragen in den Stage

```
# Eingabe
git add story1.md
git status

# Ausgabe
Auf Branch main

Noch keine Commits

Zum Commit vorgemerkte Änderungen:                                  ①
  (benutzen Sie "git rm --cached <Datei>..."
   zum Entfernen aus der Staging-Area)
    neue Datei:     story1.md                                       ②
```

① Die Dateien für den nächsten Commit
② Hier nur diese eine Datei.

Der Hinweis `git rm --cached <Datei>` bewirkt nicht das Löschen der Datei aus dem Arbeitsverzeichnis, sondern aus dem Stage!

5.1.1 Labor

Der erste Teil von *story1.md* ist nun in die Versionierung aufgenommen, es wurde aber noch kein protokollierter Punkt in der Projektgeschichte (=commit) erstellt.

Du „schreibst" an deiner Geschichte weiter und damit ändert sich der Stand zwischen Arbeitsverzeichnis und Stage. Wir schreiben nicht – wir kopieren!
Für dich bedeutet das nun das Abarbeiten folgende Schritte:

- Wechsle in den Vorlagenordner (also das Fenster)
- Checke den richtigen Stand aus: `git checkout Step_2`
- Kopiere die Datei: `cp story1.md ../buch/`
- Wechsle in den Buch-Ordner (also das Fenster)
- Checke den Status: `git status`

```
# Ausgabe
Auf Branch main

Noch keine Commits

Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git rm --cached <Datei>..."
   zum Entfernen aus der Staging-Area)
    neue Datei:     story1.md                                       ①

Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)
  (benutzen Sie "git restore <Datei>...", um die Änderungen im
   Arbeitsverzeichnis zu verwerfen)
    geändert:       story1.md                                       ②
```

① Die alte Version der Datei ist zum Commit vorgemerkt

② Die neuere Version ist noch nicht im Stage!

Wieder werden mögliche Befehle angegeben, die in diesem Zustand sinnvoll sein können!

Hinweis
Wenn du versuchst, die Datei mit `git rm --cached story1.md` aus dem Stage zu löschen, bekommst du eine Fehlermeldung. Es könnte nämlich sein dass der Vorgang zu Datenverlust führt!

```
error: die folgende Datei hat zum Commit vorgemerkte Änderungen
      unterschiedlich zu der Datei und HEAD:
         story1.md
(benutze *-f*, um die Löschung zu erzwingen)
```

Bei `git restore story1.md` gibt es keine solche Warnung! Die Datei verbleibt dabei auch im Stage unverändert erhalten, aber die Arbeitsversion wird überschrieben.

Nun kannst du erstmalig eine Differenz zwischen den Versionen der Datei erkennen:

```
git diff story1.md
```

5.1.2 Analyse der Differenz

TODO: Neu aufbauen

```
diff --git a/story1.md b/story1.md                          ①
index 9c818e7..3126a18 100644                               ②
--- a/story1.md                                             ③
+++ b/story1.md
@@ -25,3 +25,63 @@                                          ④
entdeckte sie den geheimnisvollen Tunnel.                   ⑤
  der gerade von einem seiner Flüge zurückgekehrt
  war und sich auf dem Ast des Apfelbaums niederließ.       ⑥
  +„Ich glaube, es ist ein Tunnel!" miaute Luna             ⑦
  +aufgeregt. „Vielleicht führt er zu einem verborgenen
```

① Welche Dateien werden verglichen? `a/` und `b/` sind nur Kennzeichner der Dateien. `a/` ist die ältere (z.B. Stage) und `b/` die neuere Datei (z.B. Workingdir).
② Bei 9c818e7 und 3126a18 handelt es sich um die Hash-Ids der beiden Dateien, die verglichen werden (dazu unten mehr).
③ Soll erneut klarstellen `---` ist alt, `+++` ist neu.
④ Die Zeile heißt *Hunk Header*. Er beginnt und endet mit `@@`
⑤ In den nächsten drei Zeilen gab es keine Änderungen
⑥ Das ist vor der Änderung.
⑦ Diese Zeile steht in *b* – wegen des „+“ Zeichens.

Leider ist das Format vom *Hunk Header* etwas kryptisch! Er gibt immer an, in welcher Zeile der Vergleich beginnt und wie viele Zeilen betroffen sind (vorher und nachher). Bei `@@ -3,12 +3,14 @@` beginnt der relevante Bereich ab Zeile 3 und in der ursprünglichen Version lagen 12 Zeilen vor, in der neuen Version sind es jetzt 14. Die Änderungen im Detail folgen in den nächsten Zeileen.

5.1.3 Hands on

Nun wird die Datei wieder zum Stage hinzugefügt und dann committet:

Eingabe

```
git add story1.md
git commit -m 'Guter Ansatz'
```

TODO: Hash aktualisieren

Ausgabe

```
[main (Root-Commit) 5229c6b] Guter Ansatz
 1 file changed, 4 insertions(+)
 create mode 100644 story1.md
```

5.1.4 Labor

Dieser Arbeitszyklus von *add, status, diff, commit* stellt den Kern der Arbeit mit *git* dar, zumindest wenn man alleine arbeitet. Allerdings gibt es noch weitere, sehr wichtige Techniken, die auch später bei der Arbeit im Team eine große Rolle spielen werden.

Du hast nun die Datei *story1.md* so weit fertig gestellt und möchtest an anderen Kurzgeschichten arbeiten. Du beschließt, dass fertige Geschichten und Arbeitsversionen an unterschiedlichen *Orten* aufgehoben werden sollen.

In der Softwareentwicklung sprechen wir hier von *fertigen Produkten*, die an den Kunden ausgeliefert werden und von *Experimenten, Features, Bugfixes, …*, die parallel entwickelt werden müssen. Durch ein Experiment darf die aktuell lauffähige Version Nicht beeinträchtigt werden! Du brauchst also einen *Entwicklungszweig* und einen *Auslieferungszweig*.

```
# Zweig erstellen
git switch -c entwicklung_story_2
```

Git teilt dir mit, dass du dich nun in diesem *Zweig* (=Branch) befindest. Du siehst hier immer noch deine aktuelle Story 1! Sie ist ebenfalls in dem Branch enthalten, du hast aber nicht vor, an ihr zu arbeiten. Das wäre zwar möglich, würde aber mein Szenario hier stören!

Nun beginnst du mit Story 2, indem du sie aus dem Vorlagenordner kopierst (weil wir nicht schreiben).

- Wechsle in den Vorlagenordner (Fenster)
- Checke den richtigen Stand aus: `git checkout Step_3`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner (Fenster)
- Checke den Status: `git status`

Das ist dein Einstieg in Story 2. Du bist mit deinem bisherigen Werk ganz zufrieden, hast aber zwei Ideen, wie man du weiter machen könntest. Aus diesem Grund erstellst du zunächst einen Commit

```
git add story2.md
git commit -m "Story 2 begonnen"
```

und legst für jede Idee einen Branch an:

```
git switch -c story2_idee_1
git switch entwicklung_story_2
git switch -c story2_idee_2
```

Nun nicht den Überblick verlieren! Im Branch *story2_idee_1* arbeitest du an deiner Idee 1 von *story2.md* weiter, im Branch *story2_idee_2* probierst du einen alternativen Verlauf.

Da wir nur kopieren und nicht schreiben, heißt das für dich:

- Im Buch-Ordner: `git switch stor_2_idee_1`
- Wechsle in den Vorlagenordner
- Checke den richtigen Stand aus: `git checkout Step_4`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner
- `git add story2.md`
- `git commit -m "Inhalt nach Idee1 fortgeführt"`

Da du aber noch nicht sicher bist, ob diese Version verwenden willst, arbeitest auch Idee 2 aus.

- Im Buch-Ordner: `git switch story_2_idee_2`
- Wechsle in den Vorlagenordner
- Checke den richtigen Stand aus: `git checkout Step_5`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner
- `git add story2.md`
- `git commit -m "Inhalt nach Idee 2 fortgeführt"`

Nun hast du das *Problem*, dass du drei Versionen hast:

1. Version 1 im Entwicklungszweig.
   Sie ist die Stufe vor der Neubearbeitung.
2. Version 2 im Branch für Idee 1
3. Version 3 im Branch für Idee 2

Du bist unentschieden und siehst dir die 3 Versionen noch einmal in Ruhe an.

```
git switch story2_idee_1        # in Ruhe durchlesen
git switch story2_idee_2        # in Ruhe durchlesen
git switch entwicklung_story_2  # in Ruhe durchlesen
```

Dabei fällt dir ein *grober Rechtschreibfehler* im Entwicklungszweig auf, den du sofort korrigierst. Bei uns ist es das Wort *weiße*, das sich anstelle von *weise* in den Text geschummelt hat.

- In den Vorlagenordner
- `git checkout Step_6`

- `cp story2.md ../buch/`
- Im Buch-Ordner (aktueller Branch ist *entwicklung_story_2*)
- `git add story2.md`
- `git commit -m "typo 'weiße' korrigiert"`

Beachte, dass diese Änderung in den anderen Branches *nicht* ankommt!

Du musst dich nun entscheiden, mit welcher Version du dein Projekt fortführen willst. In Wirklichkeit werden diese Branches natürlich deutlich länger sein, das Prinzip bleibt aber gleich!

Du entscheidest dich, dass du Idee 1 weiter verwenden willst und möchtest sie in den Entwicklungszweig übernehmen. In diesem Zweig solltest du aktuell gerade sein, überprüfst das aber:

```
git branch
# falls nötig
git switch entwicklung_story_2
```

Aus Sicherheitsgründen vergleichst du die Versionen kurz miteinander:

```
git diff --color-words entwicklung_story_2..story2_idee_1
```

Du siehst den Austausch des Wortes und den neu geschriebenen Text. Für *git* ist das problemlos!

Nun führst du die Zweige zusammen:

```
git merge story2_idee_1  # erster Merge
```

git erwartet von dir eine Begründung für den Merge. Gib also *Vorläufig ok* ein, speichere und schließe den Editor.

Du bist erfreut, dass dieser Merge problemlos funktioniert.

Allerdings bist du am Überlegen, ob das mit der Schildkröte so eine gute Idee ist. Du entscheidest dich, eine *Riesen*schildkröte daraus zu machen. Aus Sicherheitsgründen willst du das aber zuerst im Ideen-Zweig ändern!

Routinemäßig machst du vorher einen *diff*.

```
git diff --color-words entwicklung_story_2..story2_idee_1
```

Beachte, dass du aktuell die Dateiversionen vergleichst, die in den Commits stehen!

Den Unterschied *weise/weiße* siehst du immer noch, der ergänzte Text ist nun ja in beiden Dateien gleich.

Du wechselst in den Ideen-Branch und führst die Änderung durch. Verwende *suchen / ersetzen* oder im Terminal:

```
git switch story2_idee_1
sed -i -e "s/Schildkröte/Riesenschildkröte/g" story2.md
```

Zur Kontrolle machst du den Diff erneut:

```
# Vergleich der Banches - also das was in den letzen Commits steht
git diff --color-words entwicklung_story_2..story2_idee_1
```

Seltsam … du siehst die Riesenschildkröte nicht im Diff. Aber eigentlich ist das klar, denn es werden ja die Versionen *in den Commits* verglichen und diese neue Version ist noch nicht im Commit. Momentan haben wir also die Situation:

```
# im Branch story2_idee_1
Work:   Riesenschildkröte
Stage:  LEER
Commit: Schildkröte
```

Ein einfaches

```
git diff
```

vergleicht die *aktuelle* Version im *Working-Directory* gegen den Commit, weil der Stage aktuell leer ist (siehe folgender Abschnitt)! Hier siehst du die Riesenschildkröte und die einfache Schildkröte.

5.1.5 Genauerer Blick auf Diff

Diese Diffs sind verwirrend, betrachten wir einfachere Beispiele

Variante 1:
Im Commit, dem Stage und dem Workingdir befinden sich (unterschiedliche) Versionen einer Datei:

```
git init labor
cd labor
echo "Version commit" > datei.txt
git add datei.txt
git commit -m "Version 1"

echo "Version stage" > datei.txt
git add datei.txt

echo "Version work" > datei.txt
```

Nun wollen wir die einzelnen Versionen miteinander vergleichen. Das sind 3 Befehle:

1. work mit *stage*
   `git diff datei.txt`
   (DAS wird wichtig)
2. stage mit commit
   `git diff --cached datei.txt` oder
   `git diff --staged datei.txt`
3. work mit commit
   Wenn man den Hash (z.B. 1234) vom Commit kennt, z.B. durch
   `git log`, so kann man diesen verwenden
   `git diff 1234 datei.txt`
   Universeller ist `git diff HEAD datei.txt`

Variante 2:
Im Commit und dem Workingdir befinden sich (unterschiedliche) Versionen einer Datei. *Der Stage ist aktuell leer.*

```
git init labor
cd labor
echo "Version commit" > datei.txt
git add datei.txt
git commit -m "Version 1"

# auskommentiert
# echo "Version stage" > datei.txt
# git add datei.txt

echo "Version work" > datei.txt
```

1. work mit *commit*
   `git diff datei.txt`
   (Weil der Stage leer ist)
2. stage mit commit
   `git diff --cached datei.txt` oder
   `git diff --staged datei.txt`
3. work mit commit
   Wenn man den Hash (z.B. 1234) vom Commit kennt, z.B. durch
   \cmd{git log', so kann man diesen verwenden
   `git diff 1234 datei.txt`
   Universeller ist `git diff HEAD datei.txt`

Hier sind die Fälle 1) und 3) also identisch.
Wie kann man also sicher sein, welche Dateien ein reines `git diff` vergleicht?
Liefert `git status` einen leeren Stage, so wird mit dem Commit verglichen. Ist eine Datei im Stage, so wird gegen diese Datei verglichen.

5.1.6 Labor

Die Version mit der Riesenschildkröte willst du nun wieder in den Entwicklungsstand mergen. Dafür musst du vorher wieder einen Commit machen.

```
git add story2.md
git commit -m "Riesenschildkröte statt Schildkröte"
```

Du machst einen erneuten Vergleich der Branches mit Diff

```
# Wieder Branch-Vergleich
git diff --color-words entwicklung_story_2..story2_idee_1
```

Da du den Schreibfehler ja nur im Entwicklungszweig verbessert hast, siehst du folgende Situation:

Die *weiße Riesenschildkröte* (Schreibfehler aber richtiges Tier) im Ideen-Branch und *weise Schildkröte* (Richtige Schreibung aber falsches Tier) im Entwicklungs-Branch.
In dieser Situation ist *git* überfordert. Jeder der beiden Texte ist falsch und deshalb musst du als Benutzer entscheiden, was am Ende in der Datei stehen soll – aber das sagt dir *git* beim Merge!

```
git switch entwicklung_story_2
git merge story2_idee_1
```

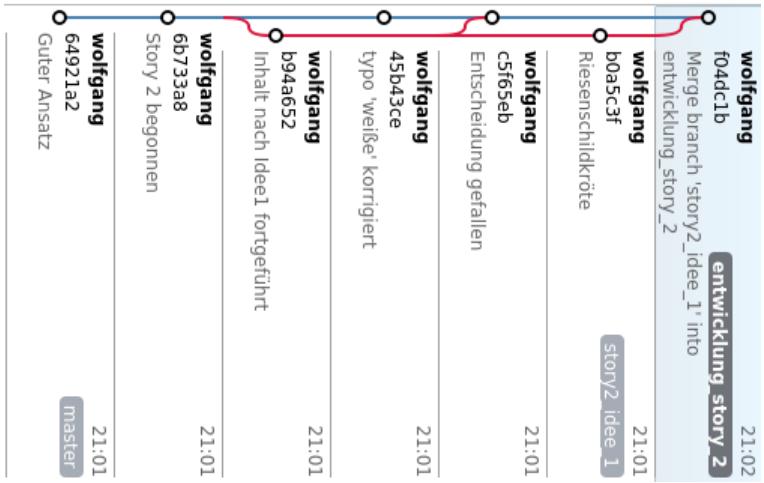
Nun siehst du eine fette Fehlermeldung, weil die Versionen nicht mehr zusammenpassen! Gerade für Anfänger ist das eine unheimliche Situation, in der man anscheinend viel kaputt machen kann … kann man ja auch. Es ist aber halb so tragisch!

Öffne die entsprechende Datei im Editor. Du erkennst sofort die Stelle, an der Änderungen erfolgt sind:

```
Freddy war nicht allein. Seine beste Freundin,
<<<<<<< HEAD
Shelly, die weise Schildkröte, lebte ebenfalls am
=======
Shelly, die weiße Riesenschildkröte, lebte ebenfalls am
>>>>>>> story2_idee_1
Teich. Shelly war für ihre ruhige und besonnene
```

Lösche einfach die Markierungen und korrigiere den Text passend.
Wieder ein *add* und ein *commit* und das Problem ist vom Tisch!

Das nachfolgende Diagramm zeigt den Ablauf der Operationen zwischen den beiden Zweigen (Idee 2 wurde ausgeblendet, weil du mit ihr bisher nichts mehr gemacht hast).



Gut - das war ein langer Weg mit vielen Abzweigungen. Es ist normal, diesen Abschnitt mehrfach lesen zu müssen!

5.1.7 Fazit

Branches erlauben gefahrloses Arbeiten an Ideen. Baut man die Szenarien nicht zu komplex auf, dann gibt auch meist keine Probleme beim Merge.

Immer wenn du Änderungen in einem Zweig vornimmst, die du auch im anderen Branch menötigst (weiße/weise) kannst du den Merge auch in die andere Richtung ausführen. Das ist ein übliches Verfahren weil man als Entwickler immer auf dem aktuellen Stand des Auslieferungszweigs sein muss, auch wenn er weiter an seinem Experiment arbeitet.

Die Vergleiche mit *diff* sind definitiv gewöhnungsbedürftig. Machst du häufige Commits auch schon bei kleinen Änderungen, so bekommst du auch übersichtliche *diffs*.

```
git diff <commit>..<commit> <datei>
```

Der Nachteil von vielen Commits sind allerdings … viele Commits. Es gibt allerdings Möglichkeiten nachträglich Commits zusammenzufassen um die Entwicklungsgeschichte zu verschlanken. Diesen Vorgang nennt man *squashen*.

6 Merge, Rebase, Squash

Im letzten Abschnitt hast du einfaches Arbeiten mit Branches gelernt und wie man sie mit Hilfe eines *Merge* zusammengeführt. Das wollen wir uns jetzt genauer ansehen. Offen geblieben sind aber folgende Fragen:

- Was passiert mit alten Branches?
- Was mache ich mit zu vielen Commits?
- Wie sieht die Alternative zu *Merge* aus?

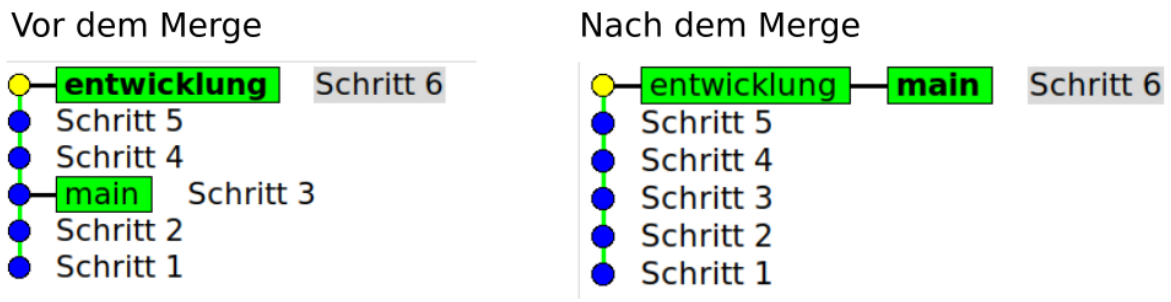
6.1 Verschiedene Merges

In der Praxis treten[1] meist zwei Fälle auf, da ich in der Regel alleine entwickle:

- Während ich am Entwicklungs-Branch arbeite, passiert auf dem Auslieferungsbranch nichts.
- Ich entwickle an einem völlig anderen Feature, das den Auslieferungsbranch gar nicht betrifft. Weil das oft aus Zeitgründen länger dauert, kann es im Auslieferungsbranch Änderungen geben, die im Entwicklungsbranch nicht interessieren.

6.1.1 Fast-Forward

Ohne Änderung im Auslieferungsbranch wird der der Entwicklungsbranch einfach (schnell davor) vor den letzten Commit des Auslieferungsbranchs gesetzt. In dieser Situation wird im Diagramm der Branch nicht einmal als visuelle Abzweigung dargestellt:

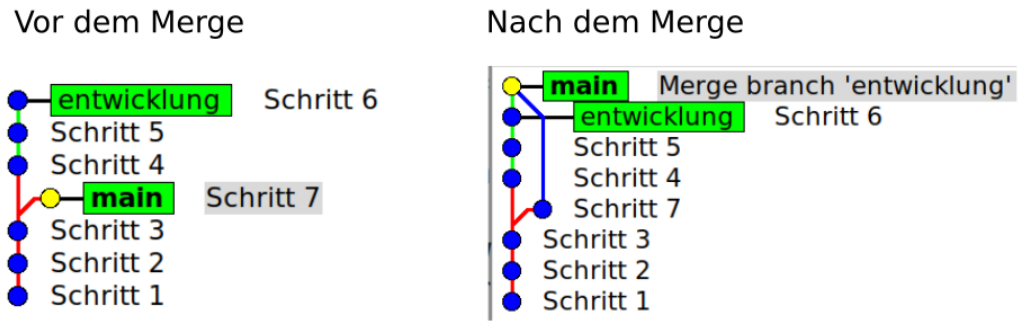


Erkennbar sind die grünen Rechtecke, die die Enden der Branches kennzeichnen. Im Bild links sind die Branches getrennt, im Bild rechts sind beide Branch-Enden nach dem Merge an der gleichen Stelle.
Wie du siehst, hat sich beim Merge die Anzahl Commits nicht verändert.

[1] Bei mir

6.1.2 Drei-Wege Merge

Wenn sich der Auslieferungsbranch parallel zum Entwicklungsbranch weiterentwickelt, sieht die Sitation ganz anders aus!



Du siehst, dass sich der *main*-Branch zu *Schritt 7* weiterentwickelt hat. Beim Merge entsteht ein neuer Commit, der die Spitze beider Branches darstellt. Du erkennst auch, dass im Diagramm die grünen Spitzen der Branches nicht am gleichen Commit sitzen!

Durch `git log --oneline --decorate --graph` (merken als „git log dog“) wird diese Sitation auch passend dargestellt:

```
*   70c1b21 (HEAD -> main) Merge branch 'entwicklung' into main
|\
| * 4a6e3a5 (entwicklung) Schritt 6
| * edaeaff Schritt 5
| * 508019b Schritt 4
* | 2ff4a45 Schritt 7
|/
* 29d2bd4 Schritt 3
* 964ee6f Schritt 2
* 66dcd4f Schritt 1
```

6.2 Alte Branches

Die Commits eines Branches stellen eine Dokumentation des Projektverlaufes dar, die für das Verständnis der erfolgten Änderungen eventuell notwendig sind. Zu viele Branches und Commits erschweren auf der anderen Seite den Überblick.

Es geht also darum, alte Branches zu löschen, die *wichtigen* Commits aber zu behalten. Für den schulischen ist das aber weniger relevant.

Um einen lokalen Branch zu löschen, gibt es folgenden Befehl:

```
git branch -d <name>
```

Er scheitert aber, wenn *git* den Branch noch nicht als abgeschlossen betrachtet.

Probieren wir das beim letzten Szenario aus. Sieh dir die Abbildung oben nich einmal an und dann lösche den Branche

```
git branch -d entwicklung
```

Du siehst eine minimale Änderung

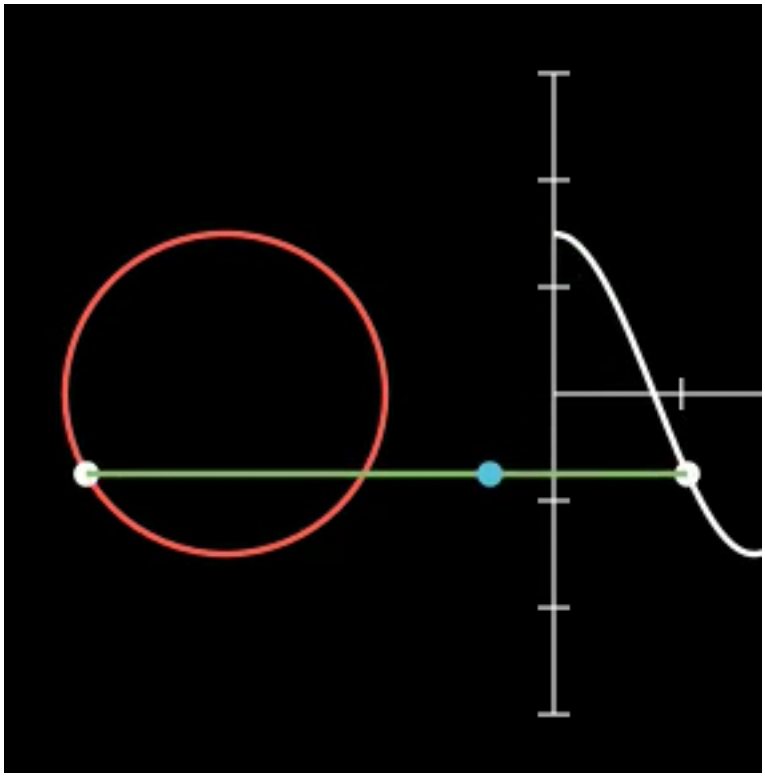
```
*   70c1b21 (HEAD -> main) Merge branch 'entwicklung' into main
|\
| * 4a6e3a5 Schritt 6   <<< hier fehlt (entwicklung)
| * edaeaff Schritt 5
| * 508019b Schritt 4
* | 2ff4a45 Schritt 7
|/
* 29d2bd4 Schritt 3
* 964ee6f Schritt 2
* 66dcd4f Schritt 1
```

Das *Etikett* vom Entwicklungsbranch wurde gelöscht, die Commits bleiben aber in exakt der gleichen Anordnung erhalten! Der Branch ist nicht mehr über seinen Namen zugänglich, die History ist aber weiterhin vorhanden.

6.3 Rebase

Neben dem *Merge* trifft man auch oft auf den *Rebase* um Branches zusammenzuführen. Er funktioniert allerdings deutlich anders und ist wegen seiner vielfältigen Möglichkeiten (z.B. Reihenfolge der Commits ändern, …) ein Werkzeug für fortgeschrittene Benutzer. Mit seiner Hilfe können auch mehrere Commits zusammengefasst werden, um die Branches zu verkürzen. Gerade bei der Zusammenarbeit im Team kann das aber sehr problematisch werden, wenn man nicht genau weiß, was man macht. Das liegt daran, dass sich die Hashwerte von Commits ändern können und wenn sich ein anderer Mitarbeiter den früheren Stand kopiert hat, dann kann das zu großem Chaos führen.

https://www.youtube.com/watch?v=CtyLg10aHN0 https://www.youtube.com/watch?v=1TNK-OkaeII



Bei einem Rebase wird der entsprechende Branch *verpflanzt* – d.h. gewissermaßen *ausgerupft* und an anderer Stelle wieder *angedockt*.

Bei diesem Vorgang werden die Hashwerte aller Commits im Branch geändert.

```
* 10f8d8d (HEAD -> arbeit) Neuer Inhalt 10
* e0ae3ce Neuer Inhalt 9
* 1eac6f3 Neuer Inhalt 8
* fa76537 Neuer Inhalt 7
* da22859 Neuer Inhalt 6
* 7903aaf Neuer Inhalt 5
* 33428f1 Neuer Inhalt 4
* 80db22a Neuer Inhalt 3
* 4a642a8 Neuer Inhalt 2
* a3645f3 Neuer Inhalt 1
| * 2c58be7 (main) Zwischenstopp
|/
* 7fea00c Start
```

Du siehst einen Branch *arbeit* mit mehreren Commits und einen neuen Commit im *main*-Branch.

Wenn du nun vom Branch *arbeit* einen Rebase auf den Branch *main* ausführst:

```
git switch arbeit
git rebase main
```

dann sieht das Ergebnis so aus:

```
* 1cad272 (HEAD -> arbeit) Neuer Inhalt 10
* 130c5b0 Neuer Inhalt 9
* 5b84ab1 Neuer Inhalt 8
* 56d74ea Neuer Inhalt 7
* 9255c1d Neuer Inhalt 6
* 412fa37 Neuer Inhalt 5
* cb91e55 Neuer Inhalt 4
* 3ab37b7 Neuer Inhalt 3
* dc4b1b0 Neuer Inhalt 2
* 9591035 Neuer Inhalt 1
* 2c58be7 (main) Zwischenstopp
* 7fea00c Start
```

Die Verzeigung ist verschwunden und der Branch *arbeit* hängt mit seinen Commits und neuen Hashwerten
am Commit mit dem Hash 2c58be7 aus dem Branch *main*.

Auch wenn es nicht so wirkt: Der Branch *arbeit* ist immer noch vorhanden! Das kannst du auch einfach testen, indem du auf den *main*-Branch wechselst und dort einen neuen Commit erstellst:

```
git switch main
echo "kontrolle" > kontrolle.txt
git add kontrolle.txt
git commit -m "Kontrolle"
```

Das Log zeigt dann

```
* 3508cce (HEAD -> main) kkkddd
| * f4ba9f7 (arbeit) kkk
| * 1cad272 Neuer Inhalt 10
....
| * dc4b1b0 Neuer Inhalt 2
| * 9591035 Neuer Inhalt 1
|/
* 2c58be7 Zwischenstopp
* 7fea00c Start
```

Die Branches laufen also wieder auseinander. Möchte man den Branch *arbeit* loswerden, dann würdest du hier eher den Rebase von *main* auf *arbeit* ausführen. *git* versucht dann, den *main*-Branch hinter den Branch *arbeit* zu hängen. Aus dieser Situation heraus kannst du dann auch den Branch *arbeit* löschen:

```
git branch -d arbeit
```

7 Arbeiten im Team

7.1 Herausforderungen

Das Arbeiten im Team stellt dich vor zwei Probleme

- Welche Server-Infrastruktur sollst du verwenden?
- Was ändert sich am Workflow in *git*

Beide Punkte sollen nachfolgend besprochen werden, wobei es zwangsläufig Überschneidungen geben wird.

7.1.1 Infrastruktur

Im Kern ist es egal ob wir vom Setup im Klassenzimmer oder im Internet sprechen – sobald ein Netzwerk beteiligt ist, sind höchstens Sicherheitsaspekte ein Thema, die Basistechnik bleibt gleich.

Für eine server-basierende Teamarbeit gibt es folgende Varianten

- einfacher *git*-Server
  Die Installation ist einfach, die Grundkonfiguration auch. Der Zugriff erfolgt über ssh. Soll das ohne Kennwort funktionieren, so müssen Schlüssel erzeugt und auf den Server eingepflegt werden. Später führt um die Schlüssel aber ohnehin kein Weg herum.
- *git*-Server mit kennwortlosem Zugriff über http.
  Funktioniert, ist aber keinesfalls für den Live-Einsatz über einen längeren Zeitraum im Internet zu empfehlen. Die entsprechenden Docker-Files befinden sich in den Materialien. (Basis: ynohat)
- Ein *git*-Server mit *Gitea*
  Gui-basiertes System mit vielen Optionen und weniger komplex als *GitLab*
- Ein *git*-Server mit *GitLab*

Bei allen Varianten ist im Prinzip eine direkte Installation auf den Server (nicht empfohlen) oder der Einsatz von Docker (sinnvoll) möglich. Die Anleitungen für die einzelnen Varianten befinden sich weiter hinten im Script.

7.1.2 Überblick

Bei der Arbeit im Team ändert sich beim lokalen Arbeiten zunächst wenig. Die Schritte *branch, add, commit, …* funktionieren wie auch bisher. Neu sind allerdings die folgenden Schritte

- Initiales Holen des Repos auf den eigenen Rechner (=clone)
- Regelmäßiges Abrufen des aktuellen Standes (=pull)
- Veröffentlichen des eigenen Standes (push)

Hierbei sind einige Spielregeln zu beachten, damit es nicht zu Problemen kommt. Diese Regeln werden meist als Workflow oder im Speziellen auch als GitFlow bezeichnet. Es gibt hier zwar gewisse – aber keinesfalls verbindliche – Standards. Jede Firma stellt für *git* ihre eigenen, fest vorgeschriebenenen Abläufe auf, an die sich die Mitarbeiter zu halten haben.

7.1.2.1 Gemeinsamkeiten

Auch die folgenden Gemeinsamkeiten müssen nicht in jeder Firma oder jedem Projekt umgesetzt werden!

Verantwortung
Im Projekt gibt es einen *main*, der aber durch entsprechende Maßnahmen für *einfache* Mitarbeiter schreibgeschützt ist. Änderungen dürfen nur nach gründlichen Tests und Code-Reviews in diesen Zweig aufgenommen werden. In der Regel gilt hier auch ein *Mehr-Augen-Prinzip* oder noch weiter gestaffelte Zuständigkeitshierarchien.

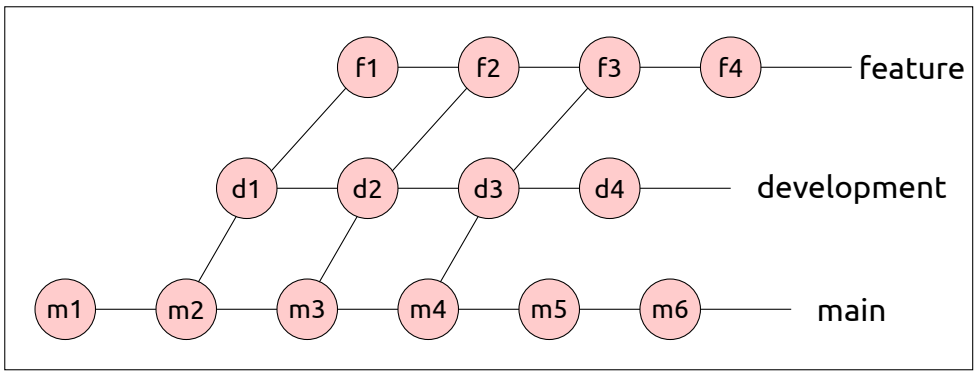
Arbeitsbranch
Je nach Größe des Projekt-Teams gibt es einen oder mehrere *development*-Branches. Die Mitarbeiter holen sich diesen Zweig und lassen von ihm ihre eigenen Arbeitszweige (Feature-Branches, Bugfix-Branches) ausgehen. Ist ihre Arbeit dort beendet, muss der Code getestet werden und dann erst erfolgt der Merge in den *development*-Branch.

7.1.2.2 Details

Da jetzt mehr Personen zu unvorhersehbaren Zeitpunkten Änderungen am Code vornehmen, muss jeder Mitarbeiter dafür sorgen, dass er immer die aktuellste Version vorliegen hat. Da er seine Entwicklung aber in seinem eigenen *Feature*-Branch vorantreibt, müssen diese Änderungen dort aber erst ankommen – dies geschieht durch Merges in die andere Richtung!

Beispiel

Aus Platzgründen zeichne ich das Branch-Diagramm hier waagrecht. Der *feature*-Branch ist auch noch nicht so weit fortgeschritten, als dass ein Merge auf den *development*-Branch erfolgt wäre.



Da noch keine wirklichen Fortschritte erzielt worden sind, dürfte der *development*-Branch auch noch nicht wieder auf den Server veröffentlicht worden sein (=push).

7.1.2.3 Pull und fetch

Die Grundidee von *holen* und *veröffentlichen* ist relativ einfach und eventuell muss man den Schülern auch nicht mehr erzählen. Da aber Code von anderen Entwicklern auf deinen Rechner kommt, solltest du diesen eventuell nicht ungesehen in deine Entwicklung aufnehmen. Der BEfehl `git pull` macht aber genau das.
Willst du an dieser Stelle sicher gehen, so machst du zuerst `git fetch` und siehst dir die Änderungen an – siehe weiter unten. Im Anschluss kannst du sie dann mit `git merge` übernehmen.

7.2 Hands on

In den nachfolgenden Abschnitten wird oft das Wort *origin* auftreten. Wir sehen uns lieber gleich an, was es damit auf sich hat. Für die einzelnen Schritte gibt es Scripts in den Kursmaterialien!

7.2.1 origin

Origin.sh

```
#!/bin/bash

# Putzen
if [ -d labor1 ]
then
  rm -rf labor1
fi

# Anlegen
mkdir labor1
cd labor1

AKTUELL=$PWD  # Pfad merken

# Repo "entfernt.git" anlegen
git init --bare entfernt.git
```

```
# branch auf main umbenennen
cd entfernt.git
git branch -m main
cd ..

# Repo in Ordner "lokal" clonen
git clone $AKTUELL/entfernt.git lokal

# Überblick
cd lokal
git status
```

Das Script `origin.sh` erstellt dir

- Ein Repository *entfernt*, das einen Server darstellt
- Ein Repository *lokal*, das deinen Rechner darstellt, indem es eine Kopie von *entfernt* erstellt (=clone).

Öffne eine *git*-Bash in einem Spielordner und führe das Script `origin.sh` aus. Die Ausgabe wird bei dir minimal abweichen.

```
Leeres Git-Repository in /tmp/labor1/entfernt.git/ initialisiert
Klone nach 'lokal' …
warning: Sie scheinen ein leeres Repository geklont zu haben.
Fertig.
Auf Branch main

Noch keine Commits

nichts zu committen (erstellen/kopieren Sie Dateien und benutzen
Sie "git add" zum Versionieren)
```

Durch

```
git remote -v
```

siehst du, woher das Repository stammt. Bei dir ist natürlich ein anderer Pfad zu sehen!

```
origin  /tmp/labor1/entfernt.git (fetch)
origin  /tmp/labor1/entfernt.git (push)
```

Wenn also in den folgenden Ausgaben und Befehlen *origin* erscheint, dann ist dieser Pfad gemeint. *Pfad* kann allerdings auch eine Netzwerkverbindung zu einem Server sein – das ist sogar der Normalfall.

Mit *origin/main* ist immer der Branch im entfernten Repository gemeint, *main* alleine bezieht sich auf den Branch in deiner lokalen Kopie.

7.2.2 Zeiger

Der nachfolgende Setup erstellt

- einen Ordner
- im Ordner ein vollwertiges Repository (–bare)

- eine lokale Kopie des Repositories mit dem Namen *susi*
- eine lokale Kopie des Repositories mit dem Namen *max*

Öffen in einem geeigneten Ordner eine *git*-Bash und führe das Script `susi_und_max.sh` aus.

Beachte, dass aus in dem Script im oberen Teil etwas Overhead nötig ist, damit die Benutzer nicht mit einem leeren Repository konfrontiert werden – dort fehlt dann nämlich ein Branch!

```
#!/bin/bash

# Putzen
if [ -d labor2 ]
then
  rm -rf labor2
fi

# Anlegen
mkdir labor2
cd labor2

git init --bare entfernt.git
cd entfernt.git
git branch -m main
cd ..

ARBEIT=$PWD  # aktuellen Pfad merken
SUSI=$ARBEIT/susi  # Abkürzungen für den Überblick
MAX=$ARBEIT/max

# Overhead
git clone $ARBEIT/entfernt vorbereitung
cd vorbereitung
echo "Hi" > README.md
git add README.md
git commit -m "Init"
git push -u origin main
cd ..
rm -rf vorbereitung

git clone $ARBEIT/entfernt susi
git clone $ARBEIT/entfernt max

#
#cd $SUSI
#echo "Hallo" > datei.txt
#git add datei.txt
#git commit -m "Hallo geschrieben"

#git branch -m main
```

Öffne ein zweites Fenster im gleichen Ordner und entscheide dich, welches für *Susi* und welches für *Max* stehen soll. Wechsle jeweils in das Repository (`cd susi` und `cd max`).

Lasse dir in beiden Repositories den Status ausgeben

```
git status
```

Es sollte jeweils erscheinen

```
Auf Branch main
Ihr Branch ist auf demselben Stand wie 'origin/main'.

nichts zu committen, Arbeitsverzeichnis unverändert
```

Im Prinzip hat Susi jetzt eine identische Kopie des Repositories erstellt. Allerdings ist der *main*-Branch jetzt *Susis main*-Branch in *diesem* Repository. Die Statusmeldung sagt dir, beide Branches *main* (Susis) und *origin/main* (der im Original Repo) sind gleich.

Nachfolgend stehen die Bezeichnungen *Original* und *SUSI* bzw. *MAX* für die entsprechenden Repositories.

Susi erstellt in *SUSI* eine Datei und einen Commit:

```
echo "Hallo Welt" >> datei.txt
git add datei.txt
git commit -m "Begrüßung"
```

und betrachtet den Status erneut:

```
Auf Branch main
Ihr Branch ist 1 Commit vor 'origin/main'.
  (benutzen Sie "git push", um lokale Commits zu publizieren)

nichts zu committen, Arbeitsverzeichnis unverändert
```

Der *git*-Ordner führt also Buch über die Beziehung zwischen den Repositories und sagt ihr, dass *SUSI* einen Commit weiter vorne ist, als das *Original*.
Das macht *git*, indem es *Pointer* auf Commits setzt (also quasi eine Datei im *.git*-Ordner führt, wo der aktuelle Hash enthalten ist.)

```
...
   config
   description
   HEAD                                                    ①
   hooks
       applypatch-msg.sample
...
   packed-refs                                             ③
   refs
       heads
           main
       remotes
           origin
               HEAD                                        ②
       tags
```

① Enthält den Hash vom letzten Commit in der lokalen Kopie
② Enthält den Hash vom letzten Commit im Original

③ Siehe nachfolgender Text

Ein `git log --oneline` zeigt dir:

```
78db34e (HEAD -> main) Begrüßung
d46681d (origin/main, origin/HEAD) Init
```

Und ein Blick in die beiden Dateien liefert die Hashes – zumindest fast, denn *git* hat hier schon wieder optimiert und den Hash vom *origin/master* in die Datei *packed-refs* umgelagert.

Das *Original* weiß von diesem Commit noch nichts – die Repositories haben ja auch noch nicht miteinander kommuniziert! Wenn Max in seinem Repository *MAX* nachsieht kann er Susis Datei nicht sehen und daran würde auch ein `git pull` nichts ändern.

Susi führt nun einen Push aus:

```
git push origin main
```

Git kommentiert das:

```
Objekte aufzählen: 4, fertig.
Zähle Objekte: 100% (4/4), fertig.
Delta-Kompression verwendet bis zu 6 Threads.
Komprimiere Objekte: 100% (2/2), fertig.
Schreibe Objekte: 100% (3/3), 292 Bytes | 292.00 KiB/s, fertig.
Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
To /tmp/labor2/entfernt
   d46681d..78db34e  main -> main
```

und jetzt liefert der Status:

```
Auf Branch main
Ihr Branch ist auf demselben Stand wie 'origin/main'.

nichts zu committen, Arbeitsverzeichnis unverändert
```

Die Zeiger in *.git* wurden also auf den gleichen Hash gesetzt.

Wechseln wir zu Max

Max hat noch sein ursprüngliches Repository und möchte auf Änderungen am Server prüfen.

```
git fetch
```

Als Ausgabe bekommt er

```
remote: Objekte aufzählen: 4, fertig.
remote: Zähle Objekte: 100% (4/4), fertig.
remote: Komprimiere Objekte: 100% (2/2), fertig.
remote: Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
Entpacke Objekte: 100% (3/3), 272 Bytes | 272.00 KiB/s, fertig.
Von /tmp/labor2/entfernt
   d46681d..78db34e  main       -> origin/main
```

Sein Status sagt ihm

```
Auf Branch main
Ihr Branch ist 1 Commit hinter 'origin/main', und kann vorgespult werden.
  (benutzen Sie "git pull", um Ihren lokalen Branch zu aktualisieren)

nichts zu committen, Arbeitsverzeichnis unverändert
```

Mit *vorspulen* ist ein Merge gemeint. Max will aber wissen *was* sich geändert hat.

```
git log origin main
```

Max sieht den Commit, der noch nicht in seinen Branch integriert ist.

```
commit 78db34e3176c4b78d4b027eac06aa4dda5c50cdb (origin/main, origin/HEAD)
Author: wolfgang <susi@t-online.de>
Date:   Wed Jan 29 12:39:38 2025 +0100

    Begrüßung

commit d46681db23c9e2b3f5379a951c026e3d673fe3e0 (HEAD -> main)
Author: wolfgang <lehrer@t-online.de>
Date:   Wed Jan 29 12:34:48 2025 +0100

    Init
```

Schön sieht man das auch mit

```
git show-ref
```

Der lokale *main* befindet sich noch bei d46681, der entfernte *origin main* hingegen bei 78db34.
Durch folgenden Befehl sieht Max die Differenz der Versionen:

```
git show 78db34
```

Geht es einfach nur um den gesamten Inhalt:

```
git show 78db34e3:datei.txt
```

Max erkennt, dass es problemlose Änderungen sind und übernimmt sie in seinen Branch:

```
git merge
```

Die Hashes ziegen, was passiert:

```
Aktualisiere d46681d..78db34e
Fast-forward
 datei.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 datei.txt
```

Hätte Max größeres Vertrauen, so würde er sich diese Recherche sparen und einfach ein `git pull` ausführen. Im Prinzip ist dieser Befehl die direkte Nacheinanderausführung von

`fetch` und `merge`.

7.2.3 Einige Fragen

Wie starte ich from Scratch
Es geht hier nicht um den Schlüsseltausch, sondern nur das Handling der Repositories.

```
# auf dem Server
git init --bare musterrepo
cd musterrepo
git branch -m main
```

```
# auf dem Client
# Besondere Schritte NUR auf dem ersten Client nötig
git clone gituser@server:port/pfad/zu/repo.git
git switch -c main
echo "Welcome" > Readme.txt
git add Readme.txt
git commit -m "initial"
git push -u origin main
```

Jeder weitere Benutzer clont nun ein funktionsfähiges Repostory.

Muss ich das 'origin main' immer angeben?

Auch wenn der Branch *main* besonders klingt, ist er doch ein ganz normaler Branch. Seine Besonderheit
ist lediglich, dass er durch das Clonen auch gleichzeitig als *upstream-branch* definiert wurde – deshalb funktioniert statt `git pull origin main` auch die kurze Version `git pull` (analog für `git push`).

Wenn du einen neuen Branch anlegst und einen commit pushen willst:

```
git switch -c probe
echo "test" > probedatei.txt
git add probedate.txt
git commit -m "Datei ergänzt"
git push
```

dann gibt es eine Fehlermeldung

```
Schwerwiegend:
Der aktuelle Branch probe hat keinen Upstream-Branch.
Um den aktuellen Branch zu versenden und den Remote-Branch
als Upstream-Branch zu setzen, benutzen Sie

    git push --set-upstream origin probe

Damit das automatisch für Branches ohne Upstream-Tracking passiert,
siehe 'push.autoSetupRemote' in 'git help config'.
```

Willst du den Branch pushen, dann musst du obigen Befehl verwenden oder kürzer

```
git push -u origin probe
```

Theoretisch geht es auch, von einem *nicht upstream* Branch direkt auf einen Upstreambranch zu pushen – das läuft aber am lokalen Branch vorbei, dem dann ein Commit fehlt:

```
git push origin probe:main
```

Ist es egal, in welchem Branch ich fetch/pull/push ausführe?
Jein.
Beim Pushen kann nicht viel schief gehen. Upstream-Branches laufen automatisch und wenn sie nicht als upstream konfiguriert sind, dann geht der Push (standardmäßig) nicht.

Beim Pull sieht es anders aus. Du kannst jeden Remote-Branch in jeden lokalen Branch *pullen* – also *mergen*. Ist der aktuelle Branch als Upstream konfiguriert, holt sich ein kurzes `git pull` die richtigen Daten. Ein `git pull <branch>` holt aber genau das, was du angegeben hast! Das ist eine Fehlerquelle!

Wie hängen lokale und remote Branches zusammen?
Zunächst sind Branches lokal und werden beim `git push` auch nicht auf den Server übertragen.

7.3 Zu klärende Fragen

Ein einfacher *git*-Server ist lediglich ein Server mit einer rudimentären Installation von *git* und einer Erreichbarkeit über das Netz. Da gerade diese Erreichbarkeit bei Windows ein Problem darstellt, handelt es sich im Regelfall um einen Linux-Server. Als solcher besitzt er üblicherweise kein grafisches Benutzerinterface (GUI).

Je nach Szenario müssen auf dem Server Benutzer angelegt und verwaltet werden. Das kann ziemlich aufwändig sein und ist bei größeren Benutzerzahlen ohne Scripting kaum machbar. Es gibt Hilfsmittel in Form von Weboberflächen, die hier aber nicht weiter thematisiert werden.

Für einen reinen *git*-Server genügt allerdings bereits ein einziger Bunutzer – z.B. *git*.

Zweck des Servers
Geht es nur um *git* oder sollen noch Wordpress, VPN-Server, … dazu kommen. Für jedes Szenario wird die endgültige Konfiguration anders aussehen …

Wer nutzt den Server
Müssen auch andere Lehrkräfte (administrativen) Zugriff auf den Server besitzen? Werden Gruppen benötigt?

Rolle der Benutzer
Sollen die Benutzer nur für die Verwendung von *git* angelegt und konfiguriert werden, oder sollen sie *vollwertige* Benutzer sein, die den Server auch in anderen Szenarien sinnvoll nutzen dürfen?

Benennung der Benutzer
Sollen die Schüler mit Zugangsdaten versehen werden, die ihren Namen widerspiegeln – Probleme siehe unten – oder sollen Zugänge im Sinne von `user1` bis `user20` verwendet werden?

Bei Verwendung von richtigen Namen entstehen folgende Probleme:

- Sonderzeichen in den richtigen Namen erschweren das automatische Erstellen von Benutzernamen.

- Verschiedene Namen können den gleichen Login ergeben.
- Behandlung von Schülern mit gleichen Namen in gleichen / verschiedenen Klassen

Die Variante mit `user1` bis `user20` ist unpersönlich, leichter einzurichten, erschwert dem Lehrer aber eventuell eine Zuordnung zum echten Schüler.

Hierfür gibt es keine perfekte Lösung.

7.4 Nur git

Wenn ein Schüler nur *git* verwenden soll, dann braucht er kein vollwertiges Benutzerkonto, an dem er sich auch für andere Arbeiten anmelden kann. Für die Verwendung von *git* genügt es bereits, wenn man einen einzigen Benutzer mit dem Namen *git* (als Beispiel) erstellt. Nachfolgend sind dann diese Schritte nötig:

- Dem Benutzer *git* wird eine *echte* Anmeldung unmöglich gemacht.
- Schüler erstellen sich SSH-Schlüssel-Paare
- Sie schicken ihren Public-Key per Mail an den Lehrer
- Der Lehrer fügt den Schlüssel in die Datei `/home/git/.ssh/authorized_keys` ein.

7.4.1 Detaillierte Anleitung

Benutzer anlegen

```
useradd -m git
passwd git  # Das Kennwort wird 2x unsichtbar abgefragt.
```

Anmeldung deaktivieren

```
nano /etc/passwd
```

Den Benutzer *git* suchen und am Ende der Zeile aus `/bin/sh` den Eintrag `/bin/false` machen. Nun mit der Tastenkombination strg + o und ‘strg + x die Änderung abspeichern.

Erstellen eines Schlüsselpaars
Unter Linux / MacOS einfach eingeben:

```
ssh-keygen -t rsa -b 4096 -C "schueler@example.com"
```

Als Dateiname *schule* vergeben und die Fragen nach einer PassPhrase beide Male mit der Enter-Taste übergehen.

Unter Windows geht das ab Windows 10 mit dem gleichen Befehl aus der *cmd.exe* oder *powershell*. Frühere Versionen müssen PuTTYgen verwenden – Anleitung z.B. über KI. Bei der Vorbereitung des Workshops hat sich das Erstellen des Schlüsselpaares hier allerdings als ziemlich problematisch erwiesen. Mit Standardnamen im Standardordner ging es problemlos, bei Angabe eines anderen Namens wurde es kompliziert.

TODO: Überarbeiten Key unter WIndows.

Zuschicken der Datei `schule.pub` per Mail
Sollte klar sein. Durch die Wahl von *schule* wird das Schlüsselpaar auch nicht im versteckten Ordner `.ssh` angelegt, sondern sichtbar direkt unter `c:\user\BENUTZER`.

TODO: Das klappt noch nicht

Kopieren des Schlüssels
Der Schlüssel muss aus der eMail in die Datei `/home/git/.ssh/authorized_keys` eingetragen werden.

TODO: Klappt das über strg + c

Die Schüler sollten nun kennwortfrei mit *git* arbeiten können.
Beim Einsatz einer Weboberfläche (später im Script) müssen die Schüler diesen Schlüssel selbst dort im System registrieren.

Hintergrund
Nach der Anmeldung über ssh wird über die Einstellungen im Benutzerkonto entschieden, welche Art von Terminal der Benutzer erhält. Üblich ist die `bash, sh, csh, ...`. Wird hier ein *ungültiges* Terminal eingetragen, so ist eine Anmeldung auf diese Weise nicht mehr möglich. Für *git* spielt diese allerdings keine Rolle, da hierfür das Terminal nicht gestartet werden muss. Da trotzdem eine Art von Anmeldung für den Zugriff auf *git* erfolgen muss, wird das Public-Key-Verfahren verwendet.

Hier muss entschieden werden, wer für die Schlüssel zuständig ist.

Varianten

- Der Lehrer erstellt Schlüsselpaare für die Schüler, die sie auf den verwendeten Geräten (Schule, Zuhause) hinterlegen und die vom Lehrer auf dem Server freigeschaltet werden. Dies kann zu Problemen führen, falls Schüler bereits aus anderen Gründen mit ssh arbeiten und deshalb schon Schlüssel besitzen[1].
- Der Schüler erstellt selbst das Schlüsselpaar oder bringt sein eigenes mit (ausdrücklich NICHT empfohlen, da der Private-Key dann auf einem Schulrechner liegt!). Er muss dann eigenständig seinen Public-Key auf dem Server hinterlegen. Das ist auch problematische, weil der Schüler zunächst das Kennwort für den Benutzer *git* besitzen muss. Nach dem Hinterlegen des Schlüssels ist es dann die Aufgabe des Lehrers, den Account zu sperren.
- Der Schüler erstellt für die Schule und zuhause unabhängige Schlüsselpaare. In diesem Fall müssen beide Schlüssel auf dem Server als gültig eingetragen werden.

7.5 Vorbereitung des Systems

Nachfolgend wird die Installation eines funktionsfähigen *git*-Servers beschrieben. Bevor man sich an die Installation auf einem Server im Internet macht, ist etwas *Training* in einer virtuellen Umgebung ganz sinnvoll. Nach Installation und Konfiguration eines Übungsservers in VirtualBox sind die Schritte identisch. Die Anleitung zur Installation von VirtualBox und dem entsprechenden Gastsystems *Ubuntu-24.04* befindet sich am Ende des Handouts. Gegebenenfalls also zuerst dort fortfahren und dann erst hierher zurückkehren.

7.6 Grundkonfiguration

Alle erforderlichen Konfigurationsschritte erfolgen über ssh im Terminal auf dem Server. In der virtuellen Maschine kann dies zwar auch direkt aus VirtualBox heraus erfolgen, zum *üben* ist es aber auch hier sinnvoll, sich mit ssh auf die VM zu verbinden.

[1] Auch das kann im Prinzip konfiguriert werden, stellt aber höhere Anforderungen an den Benutzer

7.6.1 SSH-Verbindung

Windows
Seit einiger Zeit kann auch Windows auch ohne Zusatzsoftware SSH-Verbindungen aufbauen. Allerdings unterliegt die `cmd.exe` einigen Einschränkungen (z.B. kopieren von Textinhalten), die bei der Verbindung mit Putty besser gelöst sind. Putty kann HIER als einzelne Datei heruntergeladen werden. Starten Sie dafür die Commandozeile `cmd.exe` und geben Sie folgenden Befehl ein (Benutzer und IP-Adresse müssen bekannt sein, ebenso das Kennwort):

TODO: Unterschied cmd und putty

```
ssh benutzer@ip-adresse
```

Es erscheint eine Sicherheitsabfrage, die mit `yes` und `enter` bestätigt werden muss:

TODO: Als Bild einbinden

```
The authenticity of host 'try.example.com (38.243.220.195)' can't be established.
ED25519 key fingerprint is SHA256:DSeSsfXDL2PkSlLYCt64krg9xa2vNr3og5SBJzZ/WNk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Hintergrund
Der Rechner merkt sich den Fingerabruck des Servers, zu dem die Verbindung aufgebaut wurde. Sollte später ein anderer Server diese IP-Adresse übernehmen (z.B. Man in the middle), so stimmt der Fingerabdruck nicht und es gibt eine Warnung.

Apple und Linux
Auch hier erfolgt der Verbindungsaufbau in gleicher Weise direkt aus dem Terminal.

Die Eingabe des Kennworts kann je nach Installationsvariante auch ohne sichtbare Zeichen erfolgen!

Eine ssh-Verbindung kann auf verschieden Weise wieder abgebaut werden:

- brutal, geht aber: Fenster schließen
- `exit()`
- `logout`
- strg + d

7.6.2 Software installieren

Bei *Ubuntu-24.04* ist in der Standard-Installation die Versionsverwaltung *git* bereits vorinstalliert und auch der ssh-Server sollte bereits funktionieren. Bei gemieteten Installationen muss das nicht so sein, ssh sollte aber funktionieren. Für die Verbindung mit ssh benötigt man die IP-Adresse. Von einem Mietserver kennt man die Adresse im Regelfall, in VirtualBox muss man sie erst im Terminal ermitteln, da sie vom DHCP-Server zugewiesen wird.

Hinweis
Falls man *git* im Klassenzimmer betreibt (was nicht besonders sinnvoll ist) und eine gewisse *Konstanz* in den Unterrichtsverlauf bringen möchte, dann sollte der Systemadministrator diese virtuellen Maschine in den DHCP-Server eintragen, damit sie immer die gleiche IP-Adresse bekommt!

```
ip addr show
```

Je nach Anbieter des Systems kann die Bildschirmausgabe etwas anders aussehen!

$$T \sim \sqrt{\frac{m}{D}}$$

$$\mathrm{T} \sim \sqrt{\frac{m}{D}}$$

Abbildung 7.1: Interfaces

Der veraltete Befehl `ifconfig` kann jederzeit nachinstalliert werden durch

```
sudo -i            # Admin werden
apt-get update     # Software-DB aktualisieren - dauert!
apt-get install net-tools  # mit y oder J bestätigen
```

Hinweis
Zum Installieren von Software gibt es auf Linux verschiedene Programme: *apt, apt-get, aptitude, snap, flatpak, appimages, ….* Eine detaillierte Beschreibung ginge hier zu weit. In diesem Workshop wird nur mit *apt, apt-get* und `aptitude` gearbeitet. Diese drei Programme verwenden die gleiche Datenbank für verfügbare Software und sind somit weitgehend identisch. *Aptitude* verfügt über eine – für mich – schönere Suchfunktion. Aus diesem Grund installieren wir es hier gleich nach.

```
apt-get install aptitude -y
```

Check auf installiertes git

```
git --version
```

Falls eine Fehlermeldung erscheint, muss *git* noch installiert werden:

```
apt update
apt get install git -y
```

7.6.3 Firewall

Je nach Anbieter ist auf dem Server eine Firewall aktiv oder auch nicht. Den Status erfragt man mit `ufw status`. Hierbei steht `ufw` für *uncomplicated firewall*. Bei Änderungen an der Firewall immer Vorsicht walten lassen! Man kann sich aussperren!

Ablauf

```
IN:   ufw status
OUT:  Status: inactive

IN:   ufw allow ssh
OUT:  Rules updated
      Rules updated (v6)

IN:   ufw enable
OUT:  Command may disrupt existing ssh connections.
      Proceed with operation (y|n)?
IN:   y
OUT:  Firewall is active and enabled on system startup
```

7.6.4 Ordner für Repository

Wo man diesen im Verzeichnisbaum von Linux anlegt, ist im Prinzip unwichtig – es gibt aber bessere und schlechtere Orte. Der Home-Ordner ist keine gute Idee – besser ist z.B. der Ordner `/srv`. Dort erstellen wir auch gleich ein Demo-Repository:

```
sudo -i
cd /srv
mkdir repositories
chown git:git repositories
*strg + d*
cd /srv/repositories
git init --bare demo.git
cd demo.git
git branch -m main
```

Dieses Repositoriy ist nun bereit zum Clonen.

8 gitea

Die Installation von *gitea* ist relativ einfach und kann über Kopieren und Einfügen erledigt werden. In den Kursmaterialien befinden sich die Scripte

- `setup_gitea.sh` - führt die Basisinstallation durch
- `selfreg.sh` - Selbstregistrierung ein/aus
- `tune.sh` - Feineinstellungen direkt nach der Installation

8.1 Setup-Script

In den folgenden Abschnitten sind die relevanten Passagen der Scripte dokumentiert. Die Befehle müssen nicht per Hand eingegeben werden. Das dient lediglich der Dokumentation. Die Scripte `setup_gitea.sh` und `tune.sh` müssen als *root* bzw. über *sudo* ausgeführt werden.

setup_gitea.sh

```
# Port 3000 muss für das Webinterface offen sein
ufw allow 3000/tcp

# Evtl. direkt auf der Seite https://dl.gitea.com/gitea die aktuelle
# Version checken  und hier anpassen.
wget -O gitea https://dl.gitea.com/gitea/1.22.6/gitea-1.22.6-linux-amd64

chmod +x gitea  # ausführbar machen

# Benutzer git ohne Anmeldeerlaubnis
adduser \
   --system \
   --shell /bin/bash \
   --gecos 'Git Version Control' \
   --group \
   --disabled-password \
   --home /home/git \
   git

# nötige Verzeichnisse erstellen
mkdir -p /var/lib/gitea/{custom,data,log}
chown -R git:git /var/lib/gitea/
chmod -R 750 /var/lib/gitea/
mkdir /etc/gitea
chown root:git /etc/gitea
```

```
chmod 770 /etc/gitea

# Programm an richtigen Ort verschieben
mv gitea /usr/local/bin/gitea

# Autostart einrichten

# Im richtigen Script minimiert!!!!!
cat << EOF > /etc/systemd/system/gitea.service

[Unit]
Description=Gitea (Git with a cup of tea)
After=network.target
###
# Don't forget to add the database service dependencies
###
#
#Wants=mysql.service
#After=mysql.service
#
#Wants=mariadb.service
#After=mariadb.service
#
#Wants=postgresql.service
#After=postgresql.service
#
#Wants=memcached.service
#After=memcached.service
#
#Wants=redis.service
#After=redis.service
#
###
# If using socket activation for main http/s
###
#
#After=gitea.main.socket
#Requires=gitea.main.socket
#
###
# (You can also provide gitea an http fallback and/or ssh socket too)
#
# An example of /etc/systemd/system/gitea.main.socket
###
##
## [Unit]
## Description=Gitea Web Socket
## PartOf=gitea.service
##
## [Socket]
## Service=gitea.service
## ListenStream=<some_port>
```

```
## NoDelay=true
##
## [Install]
## WantedBy=sockets.target
##
###

[Service]
# Uncomment the next line if you have repos with lots of files and get a HTTP 500 error beca
# LimitNOFILE=524288:524288
RestartSec=2s
Type=simple
User=git
Group=git
WorkingDirectory=/var/lib/gitea/
# If using Unix socket: tells systemd to create the /run/gitea folder, which will contain th
# (manually creating /run/gitea doesn't work, because it would not persist across reboots)
#RuntimeDirectory=gitea
ExecStart=/usr/local/bin/gitea web --config /etc/gitea/app.ini
Restart=always
Environment=USER=git HOME=/home/git GITEA_WORK_DIR=/var/lib/gitea
# If you install Git to directory prefix other than default PATH (which happens
# for example if you install other versions of Git side-to-side with
# distribution version), uncomment below line and add that prefix to PATH
# Don't forget to place git-lfs binary on the PATH below if you want to enable
# Git LFS support
#Environment=PATH=/path/to/git/bin:/bin:/sbin:/usr/bin:/usr/sbin
# If you want to bind Gitea to a port below 1024, uncomment
# the two values below, or use socket activation to pass Gitea its ports as above
###
#CapabilityBoundingSet=CAP_NET_BIND_SERVICE
#AmbientCapabilities=CAP_NET_BIND_SERVICE
###
# In some cases, when using CapabilityBoundingSet and AmbientCapabilities option, you may wa
# set the following value to false to allow capabilities to be applied on gitea process. The
# value if set to true sandboxes gitea service and prevent any processes from running with p
# in the host user namespace.
###
#PrivateUsers=false
###

[Install]
WantedBy=multi-user.target
EOF

# Autostart aktivieren
sudo systemctl enable gitea

# Gitea einmalig per Hand starten
sudo systemctl start gitea

echo Einrichten über Web  `http://<ip-adresse>:3000`
```

```
echo SQLITE als Datenbank einstellen
echo **WICHTIG:** Administrator einrichten!
```

8.2 Nacharbeiten

Im Nachgang sollten einige Schreibberechtigungen entzogen werden, damit nur noch *root* Änderungen vornehmen darf:

```
chmod 750 /etc/gitea
chmod 640 /etc/gitea/app.ini
```

Dieses Script deaktiviert Anmeldungen über OpenID und setzt neue Sicherheitsschlüssel.

tune.sh

```
# OpenID deaktivieren
INI="/etc/gitea/app.ini"
KEY1="ENABLE_OPENID_SIGNIN = "
KEY2="ENABLE_OPENID_SIGNUP = "
sed -i "s/${KEY1}true/${KEY1}false/g" $INI
sed -i "s/${KEY2}true/${KEY2}false/g" $INI

# Sicherheit

for KEY in INTERNAL_TOKEN SECRET_KEY JWT_SECRET LFS_JWT_SECRET
do
  VAL=$(gitea generate secret ${KEY})
  sed -i "s/^$KEY[[:space:]]*=.*/$KEY = $VAL/" $INI
done

systemctl restart gitea
```

In der Datei */etc/gitea/app.ini* kann das wieder rückgängig gemacht werden, indem der Abschnitt *[openid]* so abgeändert wird:

```
[openid]
ENABLE_OPENID_SIGNIN = true
ENABLE_OPENID_SIGNUP = true
```

Selbstregistrierung an/aus

Standardmäßig können sich die Benutzer selbst auf Gitea registrieren. Das vereinfacht die Arbeit für den Lehrer enorm, ist aber bei einer öffentlich zugänglichen Instanz problematisch. Es ist also sinnvoll, diese Einstellung nach dem erfassen der Benutzer zu deaktivieren (kann kederzeit wieder aktiviert werden).
Dazu muss in der Datei */etc/gitea/app.ini* die folgende Zeile im Abschnitt *[service]* angepasst werden:

```
ORIGINAL:  DISABLE_REGISTRATION = false
ANGEPASST: DISABLE_REGISTRATION = true
```

Dies kann über `nano /etc/gitea/app.ini` gemacht werden (Speichern: `strg + o`, Beenden `strg + x`).

Danach muss Gitea neu gestartet werden:

```
systemctl restart gitea
```

Das Setup-Script des Kurses kopiert das Script `selfreg.sh` nach */usr/local/bin*, so dass die Registrierung einfach aktiviert und deaktiviert werden kann:

```
selfreg.sh on
selfreg.sh off
```

8.3 SSH

Für den Client muss der ssh-Zugriff möglich sein (clone, push, pull). Es kommt hier der normalen ssh-Server auf Port 22 zum Einsatz[1].

Für den kennwortlosen ssh-Zugriff für clone, push, pull wird ein ein ausreichend langer ssh-Schlüssel benötigt. Die üblichen 2048 Bit genügen hier nicht! Dieser *Deploy-Key* wird später dann über die graphische Benutzeroberfläche in *Gitea* eingefügt (s.u.).

Schlüssel erzeugen

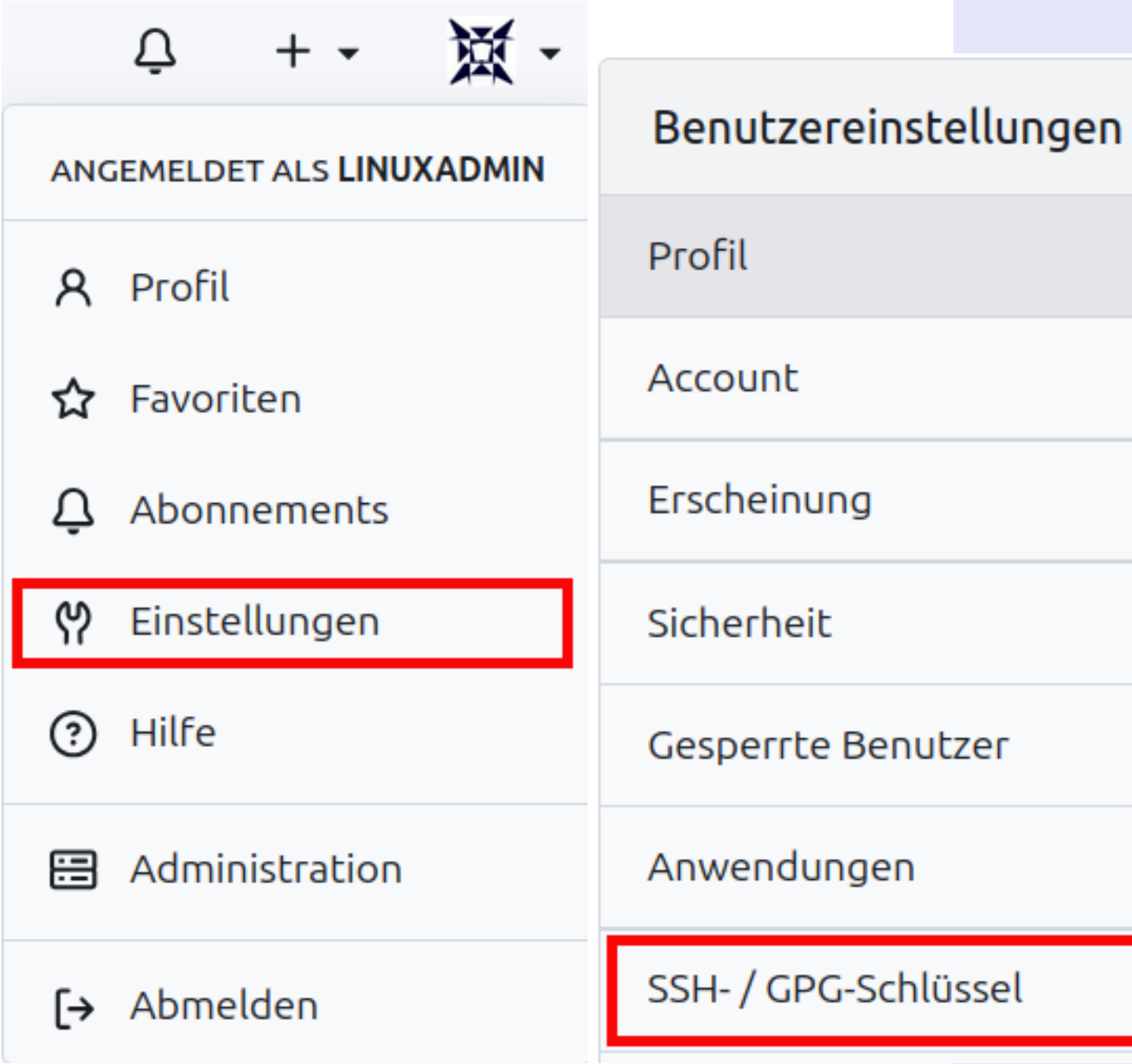
```
IN:  ssh-keygen -t rsa -b 4096
OUT: Generating public/private rsa key pair.
     Enter file in which to save the key
     (/home/linuxadmin/.ssh/id_rsa):
IN:  gitea_rsa
OUT: Enter passphrase (empty for no passphrase): ... leer lassen Enter
OUT: Enter same passphrase again: ... leer lassen Enter
OUT: Your identification has been saved in gitea_rsa
     Your public key has been saved in gitea_rsa.pub
     The key fingerprint is:
     SHA256:ty5F............Q0 linuxadmin@gitserver
     The key's randomart image is:
     +---[RSA 4096]----+
     |      . ..       |
     |.      o  .      |
     | E. . o  o o     |
     |..*. o o+ O .    |
     | =... *+SB.o     |
     | ..  o =O.o.     |
     |..     =.=.      |
     |++ .    *+       |
     |+ =.    .=+      |
     +----[SHA256]-----+
```

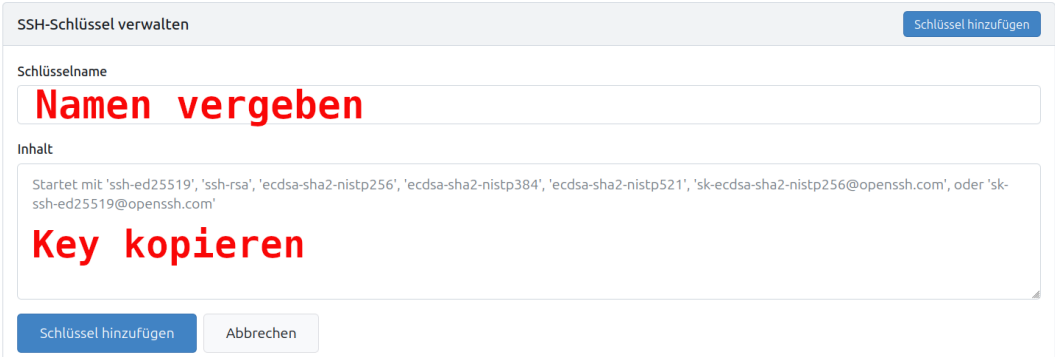
Der Inhalt von *gitea_rsa.pub* kann durch folgenden Befehl im Terminal angezeigt werden:

[1] Gitea würde über einen eigenen ssh-Server verfügen, den man in einigen speziellen Fällen verwenden könnte.

```
cat ~/.ssh/gitea_rsa.pub
```

Die mehrzeilige Ausgabe in einem Stück in die Oberfläche von Gitea kopieren:





Damit ist die Grundkonfiguration beendet. Es sollte nun noch eingestellt werden, welche Anmelde- und Registrierungsvarianten erlaubt sind, um nicht plötzlich fremde Personen in

seiner Gitea-Instanz zu haben.

8.4 HTTPS

Aktuell ist die Instanz nur über *http* und Ip-Adresse zugänglich. Es ist möglich, Gitea auch für *HTTPS* zu konfigurieren. Hierfür gibt es verschiedenen Lösungen, von denen aber eigentlich nur der Einsatz eines Reverse-Proxy mit einem entsprechenden Domain-Name sinnvoll ist. Eine derartige Konfiguration würde den Rahmen dieses Workshops aber sprengen! Zur Erinnerung: Es ging um eine möglichst einfache Installation eines funktionsfähigen Systems.

8.5 API

Oft ist es effizienter, wenn Dinge über Kommandozeile erledigt werden. Wenn z.B. 50 Benutzerkonten erstellt und mit Passwort abgesichert werden sollen, weil die Selbstregistrierung nicht gewünscht ist. In diese Fall verwendet man die API von Gitea. Dies kann von jedem Rechner aus erfolgen, auf Gitea zugreifen kann. Dafür braucht es aber noch zwei Anpassungen an der */etc/gitea/app.ini*:

1. Ergänzen Sie am Ende der Datei:

```
[api]
ENABLE_ADMIN = true
```

2. Erweitern Sie den Abschnitt *security* falls Sie die Accounts mit sehr einfachen Kennwörtern vorbelegen wollen:

```
[security]
... bisheriger Inhalt ...
PASSWORD_COMPLEXITY = off
MIN_PASSWORD_LENGTH = 6
```

Melden Sie sich als Administrator in Gitea an und rufen Sie das Menu *Einstellungen* des Benutzers auf. Unter *Anwendungen* können *Admin-Token* erzeugt werden, die einen kennwortlosen Zugriff auf das System erlauben. Stellen Sie im aufklappbaren Detail-Abschnitt ein, welche Berechtigungen erforderlich sind:

- Admin - Lesen und Schreiben
- Organisation
- Repository
- User

Kopieren Sie die lange Zahl (=Token) dann weg – sie wird nur einmal angezeigt!

Auf der Konsole (Linux / MacOS) kann dann z.B. folgender Befehl abgeschickt werden, der die Benutzerin Susi anlegt:

```
curl -X POST "http://<ip-adresse>:3000/api/v1/admin/users" \
-H "Authorization: token <kopiertes Token>" \
-H "Content-Type: application/json" \
-d '{
  "username": "susi",
```

```
  "email": "susi@sandmann.com",
  "password": "lalelu",
  "send_notify": false,
  "must_change_password": true
}'
```

Verfeinert wird dieser Befehl in ein Script verpackt – z.B. `gitea_user_anlegen.sh`, das man mit einer Namensdatei als Parameter aufrufen kann, in der folgende Spalten auftreten:

```
# login,kennwort,email  <- Diese Zeile entfernen
susi,lalelu,susi@sandmann.de
max,foobla,max@mustermann.de
...
```

8.5.1 Sinnvolle Szenarien

8.5.1.1 Organisationen

Hier bin ich mir nicht sicher, ob das kein Overkill ist. Eine Arbeit über Teams erscheint mir bei der Größe unserer Szenarien deutlich geeigneter. Das Problem ist nämlich, dass man über die API einen Benutzer nur löschen kann, wenn er in keiner Organistation mehr ist!

Es könnte sinnvoll sein, wenn sich die einzelnen Kollegen als *Organisationen* sehen. Dann müssen Schüler aber mehrfach ins System eingetragen werden und das bereits bei den eMail-Adressen Probleme.

Sinnvoller erscheint eine Namenskonvention beim Erstellen von Teams!

Organisation erstellen

```
token=1234
name=G10a
curl -X POST "http://192.168.3.195:3000/api/v1/admin/organizations" \
-H "Authorization: token ${token}" \
-H "Content-Type: application/json" \
-d '{
  "username": "${name}",
  "full_name": "${name}",
  "description": "Organisation für die Schüler der Klasse ${name}",
  "visibility": "private"
}'
```

Benutzer hinzufügen

```
token=1234
name=G10a
user=susi
curl -X PUT "http://192.168.3.195:3000/api/v1/orgs/${name}$/members/${user}" \
-H "Authorization: token ${token}"
```

User löschen

```
token=1234
user="susi"
# In welchen Organistaionen ist der User
curl -X GET "http://192.168.3.195:3000/api/v1/users/${user}/orgs" \
    -H "Authorization: token ${token}"

# Einzeln austragen, sonst ist er nicht löschbar
orga=organization1
curl -X DELETE "http://192.168.3.195:3000/api/v1/orgs/${orga}/members/${user}" \
-H "Authorization: token ${token}"

# Benutzer löschen
curl -X DELETE "http://192.168.3.195:3000/api/v1/admin/users/${user}" \
-H "Authorization: token ${token}"
```

8.5.2 Teams

Auch ein Team erfordert eine Organisation – hier würde ich einfach die Schule anlegen.

Team erstellen

```
schule="GGP"
token=1234
team=G10a
curl -X POST "http://192.168.3.195:3000/api/v1/orgs/${schule}/teams" \
-H "Authorization: token ${token}" \
-H "Content-Type: application/json" \
-d '{
  "name": "${team}",
  "description": "Team für das erste Projekt",
  "permission": "write"
}'
```

Benutzer hinzufügen

```
schule=GGP
token=1234
team=G10a
user=susi
curl -X PUT "http://192.168.3.195:3000/api/v1/orgs/${schule}/teams/${team}/members/${user}"
-H "Authorization: token YOUR_ADMIN_TOKEN"
```

TODO: GPT shgt Team_id statt Team_name!

9 Docker

Docker ist eine besonders effiziente Art der Virtualisierung. Es ist möglich, auf einem physikalischen Server viele virtuelle Server zu erstellen, diese miteinander kommunizieren und daten austauschen zu lassen und sie auch über das Internet zugänglich zu machen. Es ist also kein Problem, mehrere eigenständige Datenbankserver parallel zu betreiben und damit Übungssystem für Schüler bereit zu stellen. Wie groß der Server dimensioniert sein muss, hängt dann wieder von den speziellen Anforderungen ab (mit 6 Cores und 16 GB RAM geht schon einiges, aber sicher nicht alles!)

9.1 Idee

Nachdem Docker auf dem Server installiert ist (s.u.), kann man sich einfach ein Betriebssystem-Image aus dem Netz herunterladen und als Container starten. Natürlich ist es eine Vertrauensfrage, welche Images man verwendet. Es gibt allerdings *offizielle* Images von Firmen und Organisationen, denen man vertrauen kann:

```
docker search gitlab

NAME                            DESCRIPTION    STARS     OFFICIAL
alpinelinux/gitlab              Alpine...      11
okteto/gitlab                                  3
gitlab/gitlab-runner            GitLab ...     960
vulhub/gitlab                                  1
gitlab/gitlab-ce                GitLab...      4310
gitlab/gitlab-runner-helper                    50
gitlab/gitlab-ee                GitLab ...     549
```

In dieser gekürzten Ausgabe ist also anscheinend kein *offizielles* Image dabei. *gitlab/gitlab-ce* ist aber trotzdem offiziell. Dafür recherchiert man kurz auf dockerhub und dort ist gitlab verlinkt, wo die Informationen stehen.

Wir werden übrigens nicht *gitlab* sondern *gitea* verwenden!

Von einem Image kann man dann beliebig viele Instanzen erzeugen, die auch fast ohne Zeitverlust startbar sind. Der längste Teil ist der Download des Image.

Hintergrund
Images sind in Schichten konstruiert, die nacheinander heruntergeladen werden. Diese Schichtenstruktur ist wichtig, wenn man eigene Images erstellen möchte. Der Erstellungsprozess macht nämlich immer auf der Schicht weiter, ab der Änderungen am Image vorgenommen wurden (weitere Software, Dateiänderungen, …). Auf diese Weise wird viel weniger Zeit benötigt, als bei einer komplett neuen Erstellung!

9.2 Installation

Die Befehle befinden sich auch bei den Kursmaterialien!

TODO - Docker install Script

```
sudo -i
apt update
apt install apt-transport-https curl -y

curl -fsSL https://download.docker.com/linux/ubuntu/gpg  \
     | gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
      signed-by=/etc/apt/keyrings/docker.gpg] \
      https://download.docker.com/linux/ubuntu \
      $(. /etc/os-release && echo "$VERSION_CODENAME") \
      stable" | tee /etc/apt/sources.list.d/docker.list > /dev/null

apt update

apt install docker-ce docker-ce-cli \
    containerd.io docker-buildx-plugin \
    docker-compose-plugin -y


compose_url="https://github.com/docker/compose/releases/download"
curl -L  $compose_url/1.28.5/docker-compose-`uname \
     -s`-`uname -m` -o /usr/local/bin/docker-compose

chmod +x /usr/local/bin/docker-compose
```

Aktuell läuft Docker jetzt als Benutzer *root* (=Administrator), was man im professionellen Firmenumfeld lieber vermeidet. Es gibt Anleitung, wie man das ändert – ich selbst verwende es aber auch so.

Teste die Installation durch folgende Befehle:

```
docker --version
docker-compose --version
```

9.3 Kurzeinführung

Es gibt sehr viele verschiedene Szenarien, wie Docker verwendet werden kann und entsprechend sind auch die Befehle zahlreich und unterschiedlich kompliziert.

Nachfolgend ein kleines *Work-along*, wo einige Befehle verwendet und erklärt werden. Tipp einfach mal folgenden Befehl:

```
docker run -d -p 8090:80 --name it-tools corentinth/it-tools
```

Die Ausgabe (außer die Hashes) sollte dann so aussehen:

```
Unable to find image 'corentinth/it-tools:latest' locally
latest: Pulling from corentinth/it-tools
43c4264eed91: Pull complete
45a30f47e80f: Pull complete
4c64d3291c88: Pull complete
9dc0279166b1: Pull complete
d3b17590914c: Pull complete
50d6cfdb81c6: Pull complete
6592d833752c: Pull complete
f4cab7bcfad1: Pull complete
65e7766bfa53: Pull complete
c5f5268086b8: Pull complete
Digest: sha256:8b8128748339583ca951af03dfe02a9a4d7363f61a216226fc28030731a5a61f
Status: Downloaded newer image for corentinth/it-tools:latest
bd072e349cab6408a3b5f2d7040d586a9149107d68031089f846d0526c028f45
```

Sieht nicht spktakulär aus, bis man `http://SERVERADRESSE:8090` aufruft. Du hast hier einen kompletten Server voll mit digitalem Spielzeug erstellt - in no time!

Natürlich kannst du den Container einfach weiter laufen lassen – wir wollen uns aber einige Dinge ansehen:

Befehl	Wirkung
docker run	versucht einen Container zu erstellen
-d	im Hintergrund
-p 8090:80	Von außen 8090, im Container eigentlich Port 80
–name	sehen wir gleich
corentinth/it-tools	Name des Images

Es gibt nun zwei Stellen (oberflächlich) mit Informationen zu diesem Server.

```
docker image ls
```

Zeigt uns das heruntergeladene Image:

```
REPOSITORY              TAG       IMAGE ID       CREATED        SIZE
corentinth/it-tools     latest    bb7ba9626731   2 months ago   56.2MB
```

Mit nachfolgendem Befehl überprüft man den Status des Containers:

```
docker ps
```

Ausgabe (gekürzt)

```
CONTAINER ID   IMAGE                  COMMAND                   gekürzt ... NAMES
bd072e349cab   corentinth/it-tools    "/docker-entrypoint.…"    gekürzt ... it-tools
```

Weitere Befehle
|Befehl|Wirkung| |——|——-| |docker stop it-tools| Container stoppen| |docker stop bd072e349cab| geht auch| |docker start it-tools|Container starten| |docker start bd072e349cab| geht auch| |docker rm it-tools|Container entfernen – Image bleibt!| |docker rmi “corentinth/it-tools”|Entfernt auch das Image|

Damit ein Container gelöscht werden kann, muss er beendet sein, damit ein Image gelöscht werden kann, muss der Container gelöscht sein.

9.4 Detaillierter

Für komplexere Szenarien wird der *docker run* Befehl aber etwas unübersichtlich. Man kann nun in zwei Stufen die Komplexität erhöhen:

- Anlegen einer Datei *Dockerfile*, in der die Parameter für *docker run* stehen
- Verwenden von *docker-compose* mit einer zusätzlichen Datei *docker-compose.yml* um z.B. mehrere Server im Verbund zu managen. Dieses Verfahren werden wir bei der abschließenden, vollständigen Installation von Gitea verwenden.

Da sowohl Syntax als auch Varianten von beiden Stufen für eine schnelle Behandlung zu komplex sind, verweise ich auf entsprechende Tutorial auf youtube oder eines der vielen Bücher zum Thema – z.B. von Michale Kofler.

10 Gitea mit Docker

10.1 Basics

Wie im letzten Kapitel besprochen, wird der Einsatz von Docker durch vorgefertigte Dockerfiles und Composefiles sehr vereinfacht. Erstellt man selbst derartige Dateien, so sollte man Chatgpt nur sehr bedingt vertrauen. In seinen Kreationen tauchen oft Konfigurationsparameter auf, die es entweder gar nicht gibt, oder deren Schreibweise falsch ist. Letztendlich führt kein Weg an der Herstellerdokumentation vorbei. Die Dockerimages im Netz funktionieren aber gut, so dass man keine eigenen bauen muss.

Bei einer vollwertigen Gitea-Installation brauchen wir (mindestens) drei Container:

- Gitea
- Datenbankserver
- Runner (Erklärung folgt)

Mit dem nachfolgenden *docker-compose.yaml* können diese erstellt und konfiguriert werden.

Was ist ein Runner?
In fortgeschrittenen Szenarien kann Gitea so konfiguriert werden, dass ein Push auf einen bestimmten Branch zu nachgelagerten Aktionen auf dem Server führt. Das können Unittests sein, aber auch Compilierungen, Erstellung von Internetseiten, … was mit dem versionierten Code schlussendlich geplant ist. Wir werden unser Buch-Projekt so anpassen, dass aus unseren Dokumenten automatisch ein druckreifes PDF erstellt wird. Hier sind wir dann also bei der ursprünglichen Bedeutung der *Online-Kollaboration* angelangt.

Damit diese nachgelagerten Aktionen gestartet werden können, läuft eine *Runner*-Programm in einem eigenen Container. Dieses Programm kommuniziert mit dem Gitea-Container und führt bei Bedarf die entprechende Aktion aus. In eigentlich fast allen Fällen wird zum Ausführen ein weiterer Docker-Container erstellt, der für die eigentliche Aktion zuständig ist. Die Details dazu folgen weiter unten.

10.2 Die Dateien

Natürlich kann man für jeden benötigten Container eine eigene *docker-compose.yaml* erstellen, es ist aber sinnvoll, alle drei Container gemeinsam zu konfigurieren, da dann auch Abhängigkeiten zwischen den Containern autmatisch berücksichtigt werden. Wir besprechen die verwendete *docker-compose.yaml* zuerst Zeile für Zeile. Die vollständige Version steht am Ende des Kapitels und ist Teil der Kursmaterialien.

10.2.1 Allgemeine Einstellungen

```
version: "3"

networks:                                                    ①
  gitea:
    external: true                                           ②

volumes:                                                     ③
  gitea_data:
    driver: local
  gitea_db_data:
    driver: local
  gitea_runner_data:
    driver: local
```

① Alle drei Container müssen miteinander kommunizieren und werden deshalb in ein gemeinsames Netz mit dem Namen *gitea* eingebunden.

② Der Basis-Setup funktioniert hier auch mit *false*. Sobald der Runner aber einen neuen Container erstellt, klappt das nur noch mit *true*.

③ Die Daten der Benutzer und die Einstellungen sollen auch über Neustarts und Upgrades hinweg erhalten bleiben. Dafür dürfen sie nicht Teil des Containers sein, der dabei komplett neu erstellt wird. Es werden auf diese Weise drei Ordner mit den angegebenen Namen erstellt, die außerhalb des Containers liegen und in diesen hinein verlinkt werden.

ToDo: Was hat es genau mit dem *external:false* auf sich? ToDo: Welche andere Driver haben Volumens noch?

10.2.2 Services

```
services:
  server:
    ...
  db:
    ...
  runner:
    ...
```

Dieser schematische Aufbau beschreibt die drei anzulegenden Container. Wir besprechen die einzelnen Abschnitte nachfolgend getrennt.

10.2.3 Server

Teil 1

```
  server:
    image: docker.io/gitea/gitea:1.22.6                      ①
    container_name: gitea
    hostname: gitea
```

```
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA__database__DB_TYPE=mysql                    ②
      - GITEA__database__HOST=db:3306                     ③
      - GITEA__database__NAME=gitea                       ④
      - GITEA__database__USER=gitea
      - GITEA__database__PASSWD=gitea
```

① Hier könnte auch `gitea:latest` stehen, was Upgrades vereinfacht.
② Möglich sind auch *postgresql* und *sqlite*. Sqlite ist eine Alternative für kleine Installationen, da dabei kein eigener Datenbankserver benötigt wird.
③ Name des Datenbankservers:Port
④ Hochsicherheitskennwörter für die Fortbildung

Teil 2

```
    restart: always                                       ①
    networks:                                             ②
      - gitea
    volumes:                                              ③
      - gitea_data:/data
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro
    ports:
      - "4000:3000"                                       ④
      - "222:22"                                          ⑤
    depends_on:                                           ⑥
      - db
```

① Container immer automatisch neu starten, wenn etwas schief geht oder der Server neu gestartet wird.
② Festlegen des Netzes in dem der Container läuft
③ Die Daten von Gitea liegen im Ordner *gitea_data*, genauer unter */var/lib/docker/volumes/gitea_data*. Über Volumes wird aber auch der Zugriff auf Ressourcen des Servers gesteuert, hier der Zugriff die Eistellungen der Zeitzone. (*:ro* nur lesender Zugriff)
④ IM Container läuft Gitea auf Port 3000, der Zugriff von außen (Browser) erfolgt über Port 4000. Ist hier eher als Demo gedacht, 3000:3000 ginge auch.
⑤ Der SSH-Server (für pull, push, …) läuft im Container auf Port 22, Zugriff von außen über Port 222. Das ist nötig, da der Server (also der gemietete Server) seinen ssh-Server bereits auf Port 22 laufen hat. Das kann man zwar ändern, muss aber etwas aufpassen, dass man sich nit versehentlich aussperrt, weil man die Firewall nicht angepasst hat.
⑥ Der Container wird erst gestartet, wenn der Datenbankserver erfolgreich hochgefahren ist.

10.2.4 db

Alle relevanten Zeilen sind analog zum obigen Beispiel zu lesen.

```
  db:
    image: docker.io/library/mysql:8
    restart: always
    container_name: db
    hostname: db
    environment:
      - MYSQL_ROOT_PASSWORD=gitea
      - MYSQL_USER=gitea
      - MYSQL_PASSWORD=gitea
      - MYSQL_DATABASE=gitea
    networks:
      - gitea
    volumes:
      - gitea_db_data:/var/lib/mysql
```

10.2.5 runner

Es werden hier nur neue oder wichtige Einstellungen erklärt!

```
  runner:
    image: docker.io/gitea/act_runner:latest
    container_name: gitea_runner
    hostname: gitea_runner
    environment:
      CONFIG_FILE: /config.yaml                                  ①
      GITEA_INSTANCE_URL: "http://gitea:3000"
      GITEA_RUNNER_NAME: "Probe-runner"
      GITEA_RUNNER_REGISTRATION_TOKEN: <ausfüllen>               ②
    restart: always
    depends_on:                                                  ③
      - server
      - db
    networks:
      - gitea
    volumes:
      - ./config.yaml:/config.yaml                               ④
      - gitea_runner_data:/data
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro
      - /var/run/docker.sock:/var/run/docker.sock                ⑤
```

① Dazu kommt unten mehr!
② Füllen wir später noch aus
③ Dieser Container startet erst wenn die beiden anderen laufen
④ Ein Volume kann auch nur eine einzige Datei sein. Beachte den Unterschied bei der Schreibung von *./config.yaml* und *gitea_runner_data*. Das *./* am Anfang bedeutet, dass diese Datei im gleichen Ordner liegt wie die *docker-compose.yaml*.
⑤ Dieser Container braucht auch selbst Zugriff auf Docker, damit er selbst neue Container starten kann.

Das Config-File
Bei vielen Containern kann die Konfiguration durch Einträge in der *docker-compose.yaml*

erfolgen - wie z.B. die Kennwörter. Gerade für Kennwörter gibt es noch sicherere Varianten, das führt aber zu weit. Für den Runner werden aber noch Einstellungen nötig, die man so (anscheinend) nicht übergeben kann. Aus diesem Grund wird oben der Parameter `CONFIG_FILE` übergeben. Diese Datei befindet sich ebenfalls in den Kursmaterialien und wird hier nicht abgedruckt. Für eine frische Datei kann man folgenden Befehl aufrufen:

```
docker exec -it gitea_runner act_runner generate-config
```

oder direkt als Datei

```
docker exec -it gitea_runner act_runner generate-config > config.yaml
```

Hinweis
Vielleicht ist dir aufgafallen, dass einmal im Environment Spiegelstriche vorkommen und einmal nicht. Das ist Absicht, denn die Server wollen das unterschiedlich haben. Intern wird bei der Spiegelstrich-Variante eine Liste der Art *["key=value", "key=vlaue", …]* erstellt, die der Server verarbeiten muss. Der Runner hätte aber lieber ein Dictionary, d.h. eine Darstellung *{key:value, key:value, …}* und das passiert **ohne** die Spiegelstriche.

Das Registrierungs-Token
Bist du in Gitea eingelogged, so kannst du über das Einstellungsmenü (Das geht auf Instanz-, Repository- oder Userebene) nach *actions* suchen, dort auf *runner* und dann oben rechts auf *neuen runner anlegen* (o.ä.). Dort erscheint dann das Token, das in die *docker-compose.yaml* eingetragen werden muss. Beim Neustart registriert sich der Runner dann damit bei Gitea.

10.2.6 Starten

Damit der Runner gestartet werden kann, muss zunächst Gitea laufen – sonst kann man kein Token erstellen.

```
docker-compose up gitea db
```

Später wird der Befehl mit einem zusätzlichen `-d` gestartet, damit der Start im Hintergrund erfolgt. Ohne `-d` sieht man eventuelle Fehlermeldungen.

Nun kannst du dich bei Gitea im Browser anmelden und den ersten Benutzer einrichten, den du dann auch als Administrator verwenden kannst.

TODO: Anmeldescreen

Im Prinzip kann man auch Mailregistrierung und andere Dinge konfigurieren – für einen einfachen Betrieb genügt aber das Einrichten des Benutzers.

Nach dem oben beschriebenen Erstellen eines Tokens kannst du den Gitea und den DB-Server mit `ctrl-c` zunächst wieder beenden. Das Token wird nun in die *docker-compose.yaml* kopiert.

Nun kann auch der Runner gestartet werden. Auch jetzt zu Testzwecken zuerst ohne `-d`:

```
docker-compose up
```

Hat das ohne Fehler geklappt, dann wieder `strg + c` und dann ein richtiger Start:

```
docker-compose up -d
```

Gitea sollte jetzt verwendbar sein!

10.2.7 Einrichten

Im Prinzip ist Gitea bereits voll funktionsfähig. Wenn die Schüler **nur** über den Browser mit dem System arbeiten, sind auch keine weiteren Einstellungen nötig. Wird aber vollwertiger Zugriff (clone/pull/push) benötigt, so müssen die Benutzer noch ihren ssh-Schlüssel im System hinterlegen.

10.2.8 Schlüssel erstellen

Auf Linux und MacOS

```
ssh-keygen -t rsa -b 4096
```

Im weiteren Verlauf wird nach einer Keyphrase gefragt – leer lassen – und ein Name für den Schlüssel abgefragt – gitea passt gut.

Diesen Schlüssel lässt man sich ausgeben:

```
cat ~/.ssh/gitea.pub
```

In den Benutzereinstellungen auf Gitea kann nun ein neuer Schlüssel hinzugefügt werden.

TODO: Eventuell Screenshot

Auf Windows
… ach ja … Windows …

10.2.9 Clonen

Das Clonen eines öffentlichen Repositories über Https sollte auch ohne Schlüssel funktionieren. Für einen vollwertigen Zugriff verwendet man aber lieber *ssh* und zwar mit folgendem Befehl:

```
git clone ssh://git@mein_server:222/BENUTZER/REPO.git
```

Die Angabe von 222 ist nötig, da wir nicht Port 22 verwenden – man muss also die Adresse, die man in Gitea über dem Repository sieht, noch entsprechend anpassen.

TODO: Screenshot

11 Actions

Dieses Thema ist nun so groß und speziell, dass wir an unsere Genzen stoßen. Ich möchte nur kurz das Vorgehen für unser Szenario der Bucherstellung besprechen.

12 Clients

Generell gilt natürlich, dass im Informatikunterricht ein Tablet (oder Handy) kein geeignetes Werkzeug ist. Wir sprechen im Folgenden also von Laptops oder Desktops und diese können mit Windows, MacOS, Linux oder ChromeOS ausgestattet sein. Für jedes dieser Systeme gibt es viele verschiedene Git-Clients. Die Kommandozeile (Terminal, Bash, Git-Bash, …) ohne grafische Benutzeroberfläche ist gerade bei Anfängern nicht besonders beliebt, aber gerade im Umfeld von Servern sehr verbreitet. Entwickler, die mir den IDEs von JetBrains, VS-Code, o.ä. arbeiten, verwenden oft die dort eingebauten Git-Clients. Leider sind diese Entwicklungsumgebungen dür den Einsatz im Unterricht ein Overkill. Bei einer kleineren IDE kann ein graphischer Git-Client diese Lücke schließen.
Probleme gibt es allerdings traditionell bei der heimischen Installation der entsprechenden Software (GitKraken, …). Mit Hilfe von *portable* Versionen kann das aber deutlich vereinfacht werden. Diese Software-Variante wird nicht installiert, sondern lediglich entpackt. Der entstehende Ordner kann sogar auf einen USB-Stick ausgelagert werden. Damit sind auch die Rechner der Eltern problemlos benutzbar.

Portable Git

13 Autograding

Der Workflow für das Autograding von Schülerlösungen unterscheidet sich deutlich von der normalen Arbeit mit git/gitea, weil man den *kreativen Wissenstransfer* doch möglichst vermeiden möchte.
Beim Autograding geht es im Prinzip darum, dass die Schüler ihre Aufgaben / Prüfungen auf eine Art und Weise schreiben, dass sie mit automatisierten Test überprüft werden können. Hierbei können sichtbare Tests (der Schüler sieht sofort, ob seine Lösung passt) und versteckte Tests verwendet werden.
Technisch gesehen führt der Schüler einen Commit aus, der die Überprüfung triggert. Es ist auch möglich, dass sich der Lehrer die Schülerlösungen holt und dann erst testet.

13.1 Vorarbeiten

- Die Schüler müssen im System sein.
- Erstellen eines Teams
- Schüler dem Team zuordnen
- Ein Repository für Hausaufgaben oder spezielle Prüfungen erstellen und die benötigten Materialien hinein kopieren.

Das Script aus den Kursmaterialien führt dann für alle Teammitglieder nachfolgende Schritte aus * Das Repository für den Schüler *forken*, d.h. eine vollwertige Kopie erstellen * Die Berechtigungen so setzen, dass kein Schüler den Fork eines Mitschülers sehen kann

13.2 Workflow Schüler

Das hängt nun von der genauen Aubeitsweise ab. Im Prinzip muss der Schüler die Aufgabenstellung in der Aufgabendatei erledigen, die im Original-Repository erstellt wurde. Nach dem Erstellen der Lösung wird die Arbeit committet und auf den Server gepusht. Dort greifen dann die nachfolgende Automatismen.

Da das gleiche Repo (Hausaufgaben) über einen längeren Zeitraum genutzt werden soll, muss der Schüler regelmäßig die Änderungen im Origina-Repo in seinen Fork übertragen. Der Lehrer har dafür zu sorgen (Anleitung, …) dass es Dateien gibt (Namenskonvention), die der Schüler nicht anfasst. Auf diese Weise werden Merge-Konflikte vermieden, wenn Original-Repo und Fork zusammengeführt werden.

```
git pull


## Ablauf
```

```
* Schüler einrichten
* CI/CD System einrichten - hier wird auf jenkins verweisen? Geht auch otter-grader?
* Vorlagen für verschiedene Aufgaben erstellen ?????
* Anleitung durch README.md


## Beginn des Kurses

* Der Lehrer erstellt ein Repo
* Die Schüler forken das über die GUI
* Die Schüler Clonen ihr eigenes Repo
```

git clone https://gitea.example.com/student1/haupt-repository.git

```
* Jeden Tag wird das Repo gepullt
```

cd haupt-repository git pull origin main

```
* Die Hausaufgaben werden committed und gepusht
```bash
git add . --all
git commit -m "erledigt"
```

- Der Lehrer sammelt die Arbeien ein

```bash
#!/bin/bash

STUDENT_REPOS=("https://gitea.example.com/student1/hausaufgaben.git" "https://gitea.example.
LOCAL_DIR="student_submissions"

mkdir -p $LOCAL_DIR
cd $LOCAL_DIR

for REPO in "${STUDENT_REPOS[@]}"; do
  REPO_NAME=$(basename $REPO .git)
  if [ -d "$REPO_NAME" ]; then
    cd $REPO_NAME
    git pull origin main
    cd ..
  else
    git clone $REPO
  fi
done
```
```

14 CI/CD – Vom Commit bis zum laufenden System

Stell Dir vor, ein Entwicklerteam arbeitet gemeinsam an einer Anwendung. Mehrmals täglich werden neue Funktionen hinzugefügt, Fehler behoben oder kleine Anpassungen vorgenommen. Damit all diese Änderungen zuverlässig und ohne Chaos zusammengeführt werden können, braucht es mehr als nur gutes Teamwork – es braucht klare Prozesse und Automatisierung. Genau hier kommt CI/CD ins Spiel.

CI/CD steht für Continuous Integration und Continuous Delivery (oder Deployment) – also für die kontinuierliche Integration und Auslieferung von Software. Ziel ist es, dass Codeänderungen schnell, sicher und nachvollziehbar vom Entwicklerrechner in die fertige Anwendung überführt werden können.

Im ersten Schritt, der Continuous Integration (CI), werden alle Änderungen regelmäßig in ein gemeinsames Git-Repository eingespielt. Anstatt große, unübersichtliche Updates zu veröffentlichen, integrieren Entwickler kleine, überschaubare Änderungen. Diese werden automatisch gebaut, getestet und überprüft – so fällt sofort auf, wenn etwas nicht funktioniert. Git spielt hier eine zentrale Rolle, weil es die Grundlage für Versionskontrolle, Branches und das Zusammenführen von Code bildet.

Der zweite Schritt, die Continuous Delivery (CD), sorgt dafür, dass die geprüfte Software jederzeit bereit ist, ausgeliefert zu werden – zum Beispiel auf einem Testsystem oder sogar direkt in die Produktion. Automatisierte Prozesse übernehmen das Bauen, Testen und Bereitstellen der Anwendung. So wird vermieden, dass manuelle Schritte zu Fehlern führen oder wertvolle Zeit kosten.

CI/CD ist also weit mehr als nur ein technisches Konzept – es ist eine Arbeitsweise. Entwicklerinnen und Entwickler gewinnen durch Automatisierung Sicherheit und Freiraum: Sie können sich stärker auf den Inhalt ihrer Arbeit konzentrieren, statt auf das „Wie“ des Deployments. Teams werden dadurch effizienter, und Software kann in kleineren, stabileren Schritten wachsen.

Im Zusammenspiel mit Git entsteht so ein durchgängiger Entwicklungsfluss: Vom Commit über die automatische Prüfung bis hin zur bereitgestellten Anwendung. CI/CD bildet damit das Rückgrat moderner Entwicklungsprozesse – und ist ein Schlüsselkonzept, das jede Git-Nutzerin und jeder Git-Nutzer verstehen sollte.

Wir werden dieses Konzept nur am Beispiel eines winzigen Java-Programms durchspielen, da bereits im Vorfeld neben *git* diverse andere Voraussetzungen klären müssen:

- TDD (Test-Driven-Development)
- Arbeiten mit einem Build-System (Maven)
- Anbinden an *git* via Docker

Tutorial auf digitalocean.com

14.1 CI – Einstieg

Im Prinzip geht es um die Automatisierung von Abläufen, sobald ein bestimmter Stand in der Entwicklung erreicht ist. Wir spielen das nun an einem Minimalbeispiel durch und besprechen auch die Probleme, die (aus meiner Sicht) dabei auftreten.

Um die Abläufe einfach zu halten, arbeiten wir direkt mit *GitLab* auf der Weboberfläche – das setzt einen Account voraus!

Schritt 1
Erstellen eines leeren Repositories auf *GitLab*.
Der Name ist aktuell egal. Beim Initialisieren kann eine Readme erstellt werden – das stört nicht!

Schritt 2
Anlegen der folgenden Ordnerstruktur:

```
src/main/java/de/alp/kurs
```

Schritt 3
Erstellen der Datei *Demo.java* mit folgendem Inhalt:

```
package de.alp.kurs;
class Demo {
    public static void main(String [] args) {
        System.out.println("Willkommen zum CI/CD - Workshop!");
    }
}
```

Schritt 4
Erstellen einer Datei mit einer Datei, die unbedingt den Namen *.gitlab-ci.yml* tragen muss. Für den Anfang genügt folgender Inhalt:

```
stages:
    - build
    - test
    - deploy

ausgabe_1:
    stage: build
    script: echo "Ich bin die erste Ausgabe"
```

Ja – das hat mit Java noch nichts zu tun, das kommt aber bald. Aktuell siehst du in der *.gitlab-ci.yml* lediglich zwei Abschnitte:

- stages
- ausgabe_1

(Beachte die Konventionen für *yml* Dateien in Bezug auf die Einrückung mit Leerzeichen.)

Der Abschnitt *stages* ist verbindlich und beschreibt mögliche Namen für (Pipeline)Schritte, die abgearbeitet werden sollen. Die Namen sind frei wählbar (im Sinne gültiger Variablennamen).

Der Abschnitt *ausgabe_1* stellt einen *Job* dar, der ausgeführt wird, wenn sich das System im Pipline-Schritt *build* befindet. Es können weitere Jobs angelegt werden, die duch das Schlüsselwort *stage* den einzelnen Pipeline-Schritten zuordenbar sind.

Das Schlüsselwort *script* leitet die auszuführenden Befehle ein. Bei mehr als einem Befehl lautet die Syntax:

```
script:
    - befehl 1
    - befehl 2
    - befehl 3
```

Schritt 5
Das Speichern der Datei entspricht einem Commit und triggert dabei die Pipeline. Den Fortschritt siehst du im Menü am linken Rand im Abschnitt *Build* und dort *Pipelines*.

Klickst du auf das Symbol des laufenden (oder beendeten) Jobs, so kannst du die Bildschirmausgabe des laufenden Docker-Containers sehen.

Vorher habe ich von *Problemen* gesprochen. Je nachdem, wie schnell du das entsprechende Menü gefunden hast, lief der Job eventuell noch. Das liegt daran, dass hier standardmäßig ein komplettes Docker-Image heruntergeladen werden muss, und das dauert (unverhältnismäßig) lange für einen derartig winzigen Job. Häufige Commits führen jedes Mal zu einem Download des Docker-Images. Da in der kostenlosen Version von *GitLab* die monatlich Zeit für die Ausführung von Piplines gedeckelt ist, kann das zu Engpässen führen!

Es gibt hierfür verschiedene Lösungsansätze:

- Zeit kaufen
- *GitLab* selbst hosten
- Den Docker-Aspekt auf einen anderen Rechner auslagern.

Wenn du vor dem selbst hosten zurückschreckst, bleibt dir nur die dritte Option. Hierfür musst du einen sogenannten *Runner* und den zugehörigen *Agent* auf deinem eigenen Rechner installieren und dort auch die benötigte Infrastruktur (z.B. Docker oder die Build-Umgebung). Der Agent führt in konfigurierbaren Abständen Anfragen bei *GitLab* aus um auf das Vorhandensein von neuen Jobs zu testen. Das ist zwar etwas unelegant, löst aber das Problem der Erreichbarakeit des eigenen Rechners aus dem Internet – also der Kontaktaufnahme von Seiten von *GitLab*.

14.2 Bezug zu Java

Nun baust du die Datei *.gitlab-ci.yml* so aus, dass

- die Datei *Demo.java* beim Commit automatisch übersetzt wird
- die fertige Klasse zum Download steht

Hinweis
Natürlich fragst du dich, wofür dieser Aufwand gerechtfertigt ist, wo du das Compilieren doch einfach auf deinem Rechner ausführen kannst.
Da hast du Recht! Allerdings nur bei unserem Mini-Projekt. Wenn es um große Projekte mit mehreren Mitarbeitenden geht, die an verschiedenen Branches arbeiten, … da geht es beim Commit darum, ob z.B. das gesamte Projekt übersetzbar ist und zwar nicht in der kleines Welt des lokalen Entwicklungsrechners sondern auch auf der Zielplattform. Das

trifft im Besondern dann zu, wenn anschließend Tests ausgeführt werden und eventuell sogar die Version des Kunden automatisch ausgreollt werden soll.

Fazit: Für uns absolut überdimensioniert, im Praxiseinsatz aber absolut üblich.

Erweitere nun die *.gitlab-ci.yml* so:

```
stages:
    - build

compile-java-with-javac:
    stage: build
    image: eclipse-temurin:17-jdk
    script:
        - javac src/main/java/de/alp/kurs/*.java -d target/
        - cd target
        - java de.alp.kurs.Demo
    artifacts:
        paths: de/alp/kurs/Demo.class
```

Das verwendete Dockerimage *eclipse-temurin:17-jdk* hat einen etwas seltsamen Namen und dieser kann sich im Lauf der Zeit auch ändern. Im Zweifelsfall musst du hier etwas auf DockerHub recherchieren.

Die fertige Class-Datei wird dann als Zip-Archiv auf der Seite mit den Pipelines (rechter Rand) zum Dowmload gestellt.

14.3 Komplexere Szenarien

Sobald die Projekte umfangreicher werden, ist dieser eher manuelle Kompilieransatz und das Testen auf diesem Weg schnell zu komplex. Dafür gibt es Build-Tools wie Gradle oder *Maven*. Letzteres wollen wir näher betrachten. Für einen ersten Vorgeschmack erstellst du die Datei *pom.xml* im Haupt-Ordner (also oberhalb von *scr*):

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>de.alp.kurs</groupId>
  <artifactId>demo</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>
</project>
```

Für den Einsatz von *Maven* baust du die *.gitlab-ci.yml* so um:

```
stages:
  - build

compile-java-with-maven:
  stage: build
  image: maven:latest
```

```
  script:
    - mvn compile
    - ls target/classes/de/alp/kurs
    - cd target/classes
    - java de.alp.kurs.Demo
```

Mit dem Speichern wird das *Projekt* durch *Maven* erstellt und auch probeweise ausgeführt.

Im nächsten Abschnitt werfen wir einen genaueren Blick auf *Maven*.

14.4 Maven

14.4.1 Was ist Maven

Bei Maven handelt es sich um ein Build-Tool, wie es viele gibt und gab (z.B. Ant). Mit seiner Hilfe können bei entsprechender Konfiguration viele lästige Kleinigkeiten im Entwicklungsprozess automatisiert werden – z.B. Tests und die Verwaltung von Abhängigkeiten. Da dieses Tool bei CI/CD absolut üblich ist, brauchst du zumindest ein oberflächliches Grundverständnis.

14.4.2 Voraussetzungen

Dass ein JDK installiert sein muss, sollte klar sein, denn es soll ja mit Java programmiert werden. Natürlich gibt es wieder Unterschiede zwischen Linux, Apple und Windows.

14.4.3 Anleitung

Windows

1. **Java installieren**

Installiere das gewünschte JDK (z. B. Oracle JDK, Temurin).

2. **JAVA_HOME setzen**

   - Öffne die Systemeinstellungen: `Systemeigenschaften -> Erweiterte Systemeinstellungen -> Umgebungsvariablen`
   - Unter **Systemvariablen** auf **Neu…** klicken
     - Name: `JAVA_HOME`
     - Wert: Pfad zum JDK, z. B.: *C:*
       Program Files
       Java
       jdk-17.0.9

3. **PATH anpassen**

Systemvariable `Path` bearbeiten und hinzufügen: \datei{%JAVA_HOME%\bin}

4. **Überprüfen**

```
java -version
echo %JAVA_HOME%
```

Linux

1. **Java installieren**

Installiere ein JDK über den Paketmanager (z. B. Ubuntu/Debian):

```
sudo apt update
sudo apt install openjdk-17-jdk
```

2. **JAVA_HOME systemweit setzen**

Erstelle eine neue Datei */etc/profile.d/java.sh* mit folgendem Inhalt:

```
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
export PATH=$JAVA_HOME/bin:$PATH
```

- Datei ausführbar machen:

```
sudo chmod +x /etc/profile.d/java.sh
```

3. **Überprüfen**

```
source /etc/profile.d/java.sh
java -version
echo $JAVA_HOME
```

macOS

1. **Java installieren**

   - Lade ein JDK herunter (z. B. Temurin, Oracle) und installiere es.
   - Standard-Installationspfad: */Library/Java/JavaVirtualMachines/*.

2. **JAVA_HOME systemweit setzen**

Über `launchctl` für GUI-Apps und neue Terminalfenster:

```
sudo launchctl setenv JAVA_HOME $(/usr/libexec/java_home -v 17)
```

Optional für Shells, die beim Login starten (*/.zshrc* oder */etc/profile*):

```
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export PATH=$JAVA_HOME/bin:$PATH
```

3. **Überprüfen**

```
java -version
echo $JAVA_HOME
```

14.4.4 Installation von Maven

14.4.4.1 Linux

Eine Installation über den Paketmanager ist möglich, liefert aber wahrscheinlich eine veraltete Version. Am einfachsten dürfte dann wohl die Verwendung eines Hilfs-Scriptes sein:

```
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
```

Ab jetzt kann mit nachfolgendem Befehl genau die (aktuell) gewünschte Maven-Version installiert und verwendet werden:

```
sdk install maven 3.9.9
```

Zur Kontrolle genügt dann ein

```
mvn --version
```

(Tipp: `sdk list maven`)

Windows

TODO

Apple

TODO

14.4.5 Erster Test

(Auf Linux)

```
cd /tmp
mkdir maven-hello-world
cd maven-hello-world
mvn archetype:generate -DgroupId=com.example \
                       -DartifactId=hello-world \
                       -DarchetypeArtifactId=maven-archetype-quickstart \
                       -DinteractiveMode=false
cd hello-world/
mvn compile
mvn package
java -cp target/hello-world-1.0-SNAPSHOT.jar com.example.App
```

Wir sehen noch einige typische Warnungen, die auf eine falsche Parameterisierung in der *pom.xml* hinweisen – dazu später mehr.

14.4.6 Konventionen

Maven erwartet eine bestimmte Verzeichnisstruktur und auch entsprechende Namenskonventionen (Du musst das nicht per Hand anlegen, da gibt es Hilfs-Scripts!)

```
TREE
```

Direkt nach dem Erstellen des Projektes existieren noch nicht alle Ordner. Sie werden bei Bedarf dynmaisch von Maven angelegt.

14.4.7 Idee

Maven kennt sogenannte Livecyles. Damit sind bestimmte Abfolgen von Abläufen gemeint, die Maven ausführt.

Es gibt folgende Befehle:

- validate (macht nichts)
- compile (führt zuerst validate aus)
- test (führt zuerst validate und compile aus)
- package (erzeugt jars)
- verify (macht nichts)
- deploy (macht nicht das, nach was es klingt!)

Beim ersten Aufruf werden alle Abhängigkeiten von einem Repo im Internet in einen Cache im Home des Benutzers geladen.

Beim ersten Aufruf von *compile* wird auch das *target*Verzeichnis erstellt.

Beim Nachladen der Dependencies kann es passieren, dass das entsprechende JAR online nicht gefunden wurde (weil es nie hochgeladen wurde und deshalb einfach nicht existiert). Derartige (also eigene) jars muss man selbst in den Cache *.m2* einpflegen, damit Maven sie verwenden kann.

Im Ordner liegt eine Ordnerstruktur vor, nach producktID, Version, …, in der man selbst Ordner erstellen kann. Im Prinzip macht das auch Maven, wenn man *install* aufruft.

(https://www.youtube.com/watch?v=T00NKLQvwYE) - Windows-lastig

15 TDD - Test Driven Development

(Video mit Intellij: https://www.youtube.com/watch?v=eMU_hninZAs)

TDD ist eine sehr strenge Philosophie in Bezug auf den Ablauf bei der Software-Entwicklung. Puristen sprechen vom *Red-Green-Refactor* Zyklus, der in jedem Fall eingehalten werden muss.

15.1 Was versteht man darunter?

Grob geht es um einen Zyklus von Code schreiben, Code testen, Code optimieren, der die Entwicklung – idealer Weise automatisiert – begleitet.

Zum Beginn der Programmentwicklung wird dabei sofort der erste Test geschrieben – einer von sehr sehr vielen im Verlauf der Entwicklung. Er testet einen Programmteil, *der aktuell noch nicht existiert* und deshalb scheitern muss (Red-Phase). Nun wird die minimale Menge an Code geschrieben, damit der Test erfolgreich verläuft (Green-Phase). Nun wird das Programm optimiert (refactoring) – soweit das zu diesem Zeitpunkt bereits möglich und sinnvoll ist. Weitere Tests erzwingen Ergänzungen und Änderungen am eigentlichen Programm, so dass das Programmieren ein ständiger Wechsel zwischen den Phasen Red, Green und Refactoring wird. Viele IDEs unterstützen diesen Ablauf aktiv, erstellen also Code zum Test (oder anders herum).

Das kann kann zu Situationen führen, die für den Einsteiger in TDD befremdlich wirken. Dazu aber nach der Grundkonfiguration.

15.2 Grundkonfiguration

Wir starten mit der einfachst möglichen Konfiguration, bestehend aus

- StringFlipper.java = Java-Klasse
- StringFlipperTest.java = Testdatei
- junit5.jar

Die drei Dateien befinden sich in einem Ordner und können für den Workshop aus dem *git*-Repository, Ordner *tdd* entnommen werden. Die Zip-Datei muss dafür entpackt werden! Es entstehen die oben angeführten Dateien. Bei der Datei *junit5.jar* handelt es sich um die Version 1.10.0, die von der Maven-Seite geladen werden kann:

maven-junit

Wir arbeiten nun aus dem Ordner heraus, in dem die Dateien enpackt wurden mit dem Terminal.

```
javac -d . StringFlipper.java StringFlipperTest.java
```

Dieser Befehl compiliert die beiden Dateien und erzeugt dabei die Ordnerstruktur, die das *Package* repräsentiert:

```
.
  de
     alp
        kurs
           StringFlipper.class
           StringFlipperTest.class
```

Nun kann die Testklasse ausgeführt werden:

```
java -jar junit5.jar --class-path . --scan-class-path
```

Die Ausgabe liefert einen (absichtlich) versagenden Test und etwas Statistik:

```
Failures (1):
  JUnit Jupiter:StringFlipperTest:testSwapLastChars()
    MethodSource [className = 'de.alp.kurs.StringFlipperTest',
    methodName = 'testSwapLastChars', methodParameterTypes = '']
    => org.opentest4j.AssertionFailedError: beabsichtigt
       org.junit.jupiter.api.AssertionUtils.fail(AssertionUtils
       .java:38)
       org.junit.jupiter.api.Assertions.fail(Assertions.java:134)
       de.alp.kurs.StringFlipperTest.testSwapLastChars(
        StringFlipperTest.java:11)
       java.base/java.lang.reflect.Method.invoke(Method.java:580)
       java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
       java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

Test run finished after 92 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         1 tests found           ]
[         0 tests skipped         ]
[         1 tests started         ]
[         0 tests aborted         ]
[         0 tests successful      ]
[         1 tests failed          ]
```

In einem Produktionsumfeld sollte die Verzeichnisstruktur etwas *aufgeräumter* aussehen, damit Tests und Produktionscode voneinander sauber getrennt sind, z.B. so (wir werden später aber mit einem anderen Layout arbeiten):

```
.
  projekt
     pom.xml
     src
```

```
        main
            java
                de
                    alp
                        kurs
                            StringFlipper.java
        test
            java
                de
                    alp
                        kurs
                            StringFlipperTest.java
```

Bleiben wir aber bei unserem einfachen Lauyout und sehen uns das Testen genauer an und zwar die Testklasse. Es geht um eine Funktion, die die letzten beiden Buchstaben eines Strings vertauschen soll. Das ist zwar relativ unsinnig, dafür aber übersichtlich. Wir können sofort identifizieren, welche Fälle als Tests geprüft werden müssen:

- Ein ganz normaler String wie Hallo
- Strings der Längen 2, 1, 0

Da TDD von einer Minimalismus-Strategie ausgeht, beginnen wir mit dem leeren String. Werfen wir also einen Blick in die Testdatei:

```
package de.alp.kurs;


 import org.junit.jupiter.api.Test;
 import static org.junit.jupiter.api.Assertions.*;


public class StringFlipperTest{
   @Test
   public void testSwapLastChars(){
      fail("beabsichtigt");
   }
}
```

Der Test soll (nach TDD-Philosophie) Code testen, den es noch nicht gibt und deshalb versagen. Wir entfernen die *fail*-Zeile und ersetzen sie durch sinnvolleren Inhalt:

```
@Test
public void testSwapLastCcharsInEmptyString(){
    StringFlipper stringFlipper = new StringFlipper();
    res = stringFlipper("");
    assertEquals("", res);
    }
```

Die Ausführung des Test scheitert wie gewünscht, da die Funktion noch nicht existiert. Ändern wir das:

```
public String swapChars(String s) {
        return "";
```

Minimale Problemlösung – und wirkt wie eine *Veräppelung*. Dennoch ist das absolut ernst gemeint! Schreiben wir den nächsten Test!

```
@Test
public void testSwapLastTwoCharsOnSingleCharString(){
    StringFlipper stringFlipper = new StringFlipper();
    String res = stringFlipper.swapChars("A");
    assertEquals("A",res);
    }
```

Wir können den Test laufen lassen und beobachten sein Scheitern. Die Funktion muss also verfeinert werden. Beim aktuellen Stand wäre das z.B. so möglich:

```
public String swapChars(String s) {
    return s;
```

Beide Tests liefern grünen Status! Probieren wir einen String mit zwei Buchstaben.

```
 @Test
 public void testSwapLastTwoCharsOnTwoCharString(){
    StringFlipper stringFlipper = new StringFlipper();
    String res = stringFlipper.swapChars("AB");
    assertEquals("BA",res);
    }
```

Der Test scheitert und wir verfeinern die Funktion erneut. Da es sich um eine neue Situation handelt, trennen wir die Fälle:

```
public String swapChars(String s) {
    int length = s.length();
    if (length < 2) {
        return s;
        }
    StringBuilder sb = new StringBuilder();
    sb.append(2);
    sb.append(1);
    return sb.toString();
```

Und wieder *grün*! Nun wollen wir einen vollwertigen String probieren!

```
@Test
    public void testSwapLastTwoCharsOnNormalString(){
    StringFlipper stringFlipper = new StringFlipper();
    String res = stringFlipper.swapChars("Hallo");
    assertEquals("Halol",res);
    }
```

Wieder zum Verfeinern:

```
public String swapChars(String s) {
    int length = s.length();
    if (length < 2) {
        return s;
```

```
        }
    StringBuilder sb = new StringBuilder();
    for (int i=0; i < length-2; i++){
        sb.append(s.charAt(i));
    }
    sb.append(s.charAt(length - 1));
    sb.append(s.charAt(length - 2));
    String res = sb.toString();
    return res;
    }
```

Damit haben wir alle Fälle abgehakt, die uns zum Testen eingefallen sind. Ja – dieser Prozess ist langatmig und verführt zum Verwenden von Abkürzungen. Leider führen diese Abkürzungen am Ende zu schlecht programmierter Software! Ein Beispiel ist der Computer der Bosch-eBikes. Hier ist es möglich, dass negative Fahrzeiten auftreten, wenn man keine regelmäßigen Resets der Daten durchführt – ein Fall, der bei gutem Testing aufgefallen wäre.

Da Testing so wichtig ist, wird es mittlerweile von allen IDEs unterstützt und Build-Tools wie Maven haben es ebenfalls integriert. Für unser eigentliches Thema CI/CD werden wir aber gleich *Maven* verwenden und uns deshalb vom manuellen Testen weitgehend verabschieden.

```
public class StringFlipperTest{
   @Test
   public void testSwapLastChars(){
      StringFlipper stringflipper = new StringFlipper();
      assertEquals(1, 1);
   }
}
```

Rerun – definitiv Red, da es die Klasse noch nicht gibt – also erstellen wir sie im Ordner *src/java/de/alp/kurs*

```
nano ../java/de/alp/kurs/StringFlipper.java
```

Der Aufruf zum Compilieren muss nun allerdings einen Level nach oben springen!

```
javac ../java/de/alp/kurs/StringFlipper.java
```

15.2.1 Gewohnheiten

- Zuerst das Kriteium formulieren und davon rückwärts arbeiten
- Jeder Test muss zuerst gegen einen definierten Fail laufen.
- Ein Test bekommt nur einen Grund zum Versagen

15.3 Java

15.3.1 Idee

Im Zusammenhang mit *git* könnte ein Projekt für den Unterricht die Entwicklung einer Krypto-Software sein, bei der der Benutzer mit verschiedenene Methoden (Cäsar, Vigenere, …) Klartext in verschlüsselten Text transformieren kann. Zentral wird eine kleine GUI bereitgestellt oder entwickelt, in die die Module der Projektgruppen eingebunden werden können.

Nachfolgend nur kurz das Vorgehen für einen einfachen Cäsar mit GUI-Anbindung. Wem dies eine zu geringe Anforderung ist, kann auch gerne eine andere Technik umsetzen – schlussendlich geht es eigentlich um die Verwendung von Maven in diesem Zusammenhang.

Natürlich kann jede aktuelle IDE auf Knopfdruck auch Maven-Projekte generieren, wir wollen es hier aber *from scratch* durchführen, um die Vorgänge hinter den Kulissen besser zu verstehen. Das ist u.a. deshalb nötig, weil für CI/CD Konfigurationsschritt nötig sind, die diesen hier ziemlich genau entsprechen.

15.3.2 Erstellen des Projektes

Im Terminal wird bei bereits installiertem Maven eine korrekte Ordnerstruktur initialisiert.

```
mvn archetype:generate  -Dfilter=quickstart
```

Es müssen einige Parameter eingetellt werden:

```
Version: 1.5 (Taste 9)
groupID: de.alp.kurs
artifactID: krypto
version: 1.0-SNAPSHOT
package: de.alp.kurs
Bestätigen: Y
```

Das liefert folgende Struktur:

```
.
  krypto
      pom.xml
      src
          main
              java
                  de
                      alp
                          kurs
                              App.java
          test
              java
                  de
                      alp
                          kurs
```

AppTest.java

Dieses Projekt kann bereits (fast) compiliert werden. Zuerst müssen wir allerdings noch einige Einträge in der *pom.xml* angepasst werden:

- die lokal installierte Java-Version in die *pom.xml* eintragen. Default ist 17, ich verwende 21.
- Konfiguration des *jar*-Plugin:

```
...
<plugin>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.4.2</version>
    <configuration>
        <archive>
            <manifest>
                <mainClass>de.alp.kurs.App</mainClass>
            </manifest>
        </archive>
    </configuration>
    </plugin>
...
```

Mit `mvn package` komplieren wir, führen einen Test aus und erstellen das passende jar-Archiv.

```
java -jar target/krypto-1.0-SNAPSHOT.jar
```

16 cicd Buch

16.1 Gitlab Runners

Runners are small programs that are installed separately from the main GitLab application. Their purpose is to receive new CI/CD jobs published by GitLab and follow the jobs' instructions as specified in the .gitlab-ci.yml file. Runners can be installed and configured to work with a variety of types of infrastructure, including standalone servers, VMs, containers, and others.

Zusätzlich zu einem GitLab-Account braucht man ein Gerät, auf dem dem der GitLab Runner (Agent) läuft. Das kann *jeder* Rechner sein (Auch der Server)!

CICD Pipelines laufen NICHT innerhalb von GitLab, da jeder Job ganz individuelle Einstellungen erfordert.

Runners akzeptieren Jobs von GitLab und liefern das Ergebnis zurück an GitLab.

Was der Runner macht, ist in der .gitlab-ci.yml definiert.

Jeder Commit triggert einen neuen Lauf der Pipeline.

Bekommt ein Runner einen Job zugewiesen, so holt sich der Runner den Commit aus dem Repo.

16.1.1 Terminiologie

GitLab Runner: Die Runner-Software auf einem Rechner
Runner: Das sind quasi registreierte Instanzen, die dann Jobs abarbeiten.
Im Prinzip zur Laufzeit ein Runner-Prozess, der einen Job abarbeitet.

Die Gitlab Runner - Software übernimmt die Jobs von GitHub und startet daraufhin die passenden Runner-Anwendungen.

16.1.2 Runner Agent

Eine Art von Relais, wenn der runner nicht direkt über einen anstehenden Job informiert werden kann. Der Agent pollt in konfigurierbaren Abständen den GitLab-Server.

Runner können auf verschiedenen Leveln in GitLab registriert werden. Das schließt Projekte, Gruppen, … ein.

Shared Runners Only Apply to Self-Managed GitLab

Administrators can only configure shared runners on self-managed GitLab instances. GitLab.com customers can choose to use SaaS runners provided by GitLab, as well as register their own group or specific runners. A certain number of pipeline minutes for SaaS runners

are included in each GitLab license tier, with additional minutes available for purchase. There is never a charge to use your own group or specific runners.

16.1.3 Mögliche Executors

16.1.3.1 Docker

In diesem Fall muss Docker auf dem gleichen Rechner wie der GitLab Runner installiert sein. Meistgenutzter Typ auch bei den SaaS Runners von GitLab. Perfekt für reproduzierbare Umgebungen.

Welches Image verwendete wird, kann auf verschiedenen Ebenen festgelegt werden (default, pipeline, job).

Das Image kann nicht nur auf Dockerhub gehostet sein!

16.1.3.2 Shell

Stellt im Prinzip eine Shell-Session auf dem Host dar, auf dem der GitLab-Runner installiert ist. Im Prinzip wird ein Keyboard simuliert.

Hier bekommt man leider keine *suabere* Umgebung und es können Reste vom letzten Lauf zurückbleiben, die den Ablauf stören. Besser Finger weg.

16.1.3.3 Hypervisoren

VB, Parallels, Kubernetes, …
Für spezielle Anwendungen – spezieller als unsere Szenarien.

16.1.3.4 Docker Machine

Simuliert einen kompletten Docker-Host und nicht nur einen Container. Wird von Docker nicht mehr aktiv weiter entwickelt.

16.1.3.5 SSH

Damit können Remote-Befehle auf anderen Rechnern ausgeführt werden. Wir sprechen von „bash-Level“. Kann sinnvoll sein, um auf einen Server zu grätschen, auf dem man keinen Runner installieren will / kann.

16.2 Runner Tags

Ein Runner kann mit einem Tag versehen werden, analog zu Jobs. Damit passen Job und Runner zusammen.

```
deploy-to-staging:
    stage: staging
    script: ./deploy-staging.sh
    tags:
        - windows
```

```
        - staging
```

Dieser Job kann nur (default) von Runnern ausgeführt werden, die ebenfalls die Tags *windows* und *staging* besitzen. Das kann in den Einstellungen eines Runners aber deaktiviert werden. (Vorsicht: Runner-Tags haben nichts mit Git-Tags zu tun)

17 Introduction

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

18 Summary

In summary, this book has no content whatsoever.

References

Knuth, Donald E. 1984. „Literate Programming“. *Comput. J.* 27 (2): 97–111. https://doi.org/10.1093/comjnl/27.2.97.